

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



Báo Cáo Bài Tập Lớn

Đề tài: **Hạ Tầng Khóa Công Khai**
(Public Key Infrastructure – PKI)

Học phần: **Nhập môn An toàn thông tin – IT4015**

Mã học phần: IT4015

Mã lớp: 167834

Giảng viên:	PGS. TS. Nguyễn Linh Giang	
Nhóm sinh viên:	Đỗ Trường Giang	20235066
	Hoàng Minh Khôi	20230042
	Phạm Gia Hưng	20230036
	Nguyễn Thanh Tú	20230075

Hà Nội, June 26, 2026

Contents

1	Giới thiệu	1
1.1	Vấn đề đặt ra	1
1.2	Phạm vi và bố cục báo cáo	2
2	Cơ sở Mật mã học và Sự tiến hóa của Hệ thống Khóa	4
2.1	Mật mã đối xứng và hạn chế	4
2.2	Mật mã khóa công khai	7
2.3	Tại sao mật mã bất đối xứng chưa đủ?	19
3	Cấu trúc hạ tầng khóa công khai PKI	22
3.1	Các mô hình phân phối khóa công khai	22
3.1.1	Bài toán phân phối khóa công khai	22
3.1.2	Public Announcement	22
3.1.3	Host-based Trust	23
3.1.4	Web of Trust (PGP)	23
3.1.5	Hierarchical PKI	23
3.1.6	So sánh trade-off	23
3.2	Định nghĩa và mục tiêu PKI	24
3.3	Các thành phần PKI	25
3.4	Chu kỳ sống của chứng chỉ	27
4	Chứng chỉ số và các chuẩn	29
4.1	Chứng chỉ số X.509	29
4.2	Các trường X.509 v3	31
4.3	Distinguished Name	31
4.4	Extension quan trọng	32
4.5	Certificate Signing Request	33
4.6	Phân loại chứng chỉ	33
4.7	Các chuẩn PKCS	34
4.8	Các chuẩn hóa và quy tắc vận hành	34

5	Triển khai và ứng dụng thực tế	36
5.1	HTTPS / TLS	36
5.2	Chữ ký số văn bản	38
5.3	S/MIME	38
5.4	VPN và IPSec	38
5.5	Code Signing	39
5.6	eGovernment Việt Nam	39
6	Các hệ thống PKI mã nguồn mở	41
6.1	Tiêu chí phân loại	41
6.2	OpenSSL	42
6.3	XCA	43
6.4	EJBCA	43
6.5	Let's Encrypt và Certbot	44
6.6	Vault PKI	44
6.7	OAuth/OIDC và PKI	45
6.8	Dogtag và FreeIPA	45
6.9	So sánh hệ sinh thái	45
7	Thách thức và xu hướng	47
7.1	Thách thức hiện tại	47
7.2	Tự động hóa và giám sát	47
7.3	PKI trong hệ thống hiện đại	48
7.4	Sẵn sàng cho kỷ nguyên hậu lượng tử (Post-quantum)	49
8	Kết luận	50
8.1	Tổng kết	50
8.2	Ý nghĩa và lời cảm ơn	50
	Tài liệu tham khảo	51

Chapter 1

Giới thiệu

1.1 Vấn đề đặt ra

Mật mã khóa công khai (public-key cryptography) đã mang lại một bước tiến quan trọng trong lĩnh vực an toàn thông tin khi cho phép các thực thể trao đổi dữ liệu bí mật và thực hiện chữ ký số mà không cần phải chia sẻ khóa bí mật từ trước. Nhờ đó, các hệ thống có thể hoạt động trên môi trường mạng mở như Internet mà vẫn đảm bảo được tính bảo mật và toàn vẹn của dữ liệu.

Tuy nhiên, một vấn đề cốt lõi vẫn chưa được giải quyết bởi bản thân mật mã khóa công khai, đó là: làm thế nào để xác định rằng một khóa công khai thực sự thuộc về đúng thực thể mà nó tuyên bố đại diện. Nói cách khác, hệ thống vẫn thiếu một cơ chế để ràng buộc giữa danh tính (identity) và khóa công khai (public key).

Trong môi trường mạng không tin cậy, kẻ tấn công có thể khai thác điểm yếu này để thực hiện tấn công trung gian (Man-in-the-Middle). Ví dụ, khi hai bên A và B trao đổi khóa công khai, kẻ tấn công có thể chặn kết nối và thay thế khóa công khai của B bằng khóa của chính mình. Khi đó, A sẽ mã hóa dữ liệu bằng khóa của kẻ tấn công mà không hề hay biết. Mặc dù các thuật toán mật mã vẫn hoạt động chính xác về mặt kỹ thuật, toàn bộ thông tin trao đổi đã bị lộ và có thể bị chỉnh sửa trước khi đến đích.

Một ví dụ thực tế có thể thấy trong quá trình truy cập website. Nếu không có cơ chế xác thực đáng tin cậy, người dùng có thể bị chuyển hướng đến một website giả mạo có giao diện giống hệt website thật. Website giả này cung cấp một khóa công khai do kẻ tấn công kiểm soát, khiến người dùng vô tình thiết lập kết nối “an toàn” với chính kẻ tấn công.

Do đó, một yêu cầu quan trọng được đặt ra là cần có một cơ chế đáng tin cậy để xác minh rằng một khóa công khai thực sự thuộc về đúng thực thể tương ứng. Cơ chế này phải hoạt động hiệu quả ngay cả trong môi trường mạng mở, nơi không thể tin tưởng các kênh truyền thông. Hạ tầng khóa công khai (Public Key Infrastructure – PKI) được đề xuất nhằm giải quyết chính vấn đề này, bằng cách cung cấp một hệ thống quản lý và xác thực khóa công khai dựa trên các bên thứ ba tin cậy.

Hình 1.1 minh họa kịch bản tấn công trung gian khi hai bên chỉ trao đổi khóa công khai trực tiếp. Hình 1.2 tóm tắt cách CA đóng vai trò neo tín nhiệm, giúp client xác minh khóa thuộc về đúng chủ thể thay vì tin vào kênh truyền.

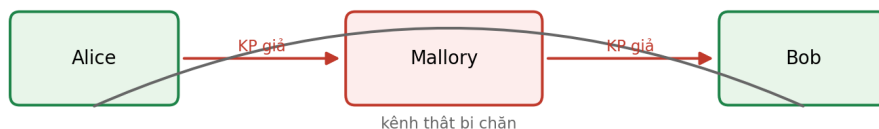


Figure 1.1: Tấn công trung gian khi phân phối khóa công khai không có bên tin cậy

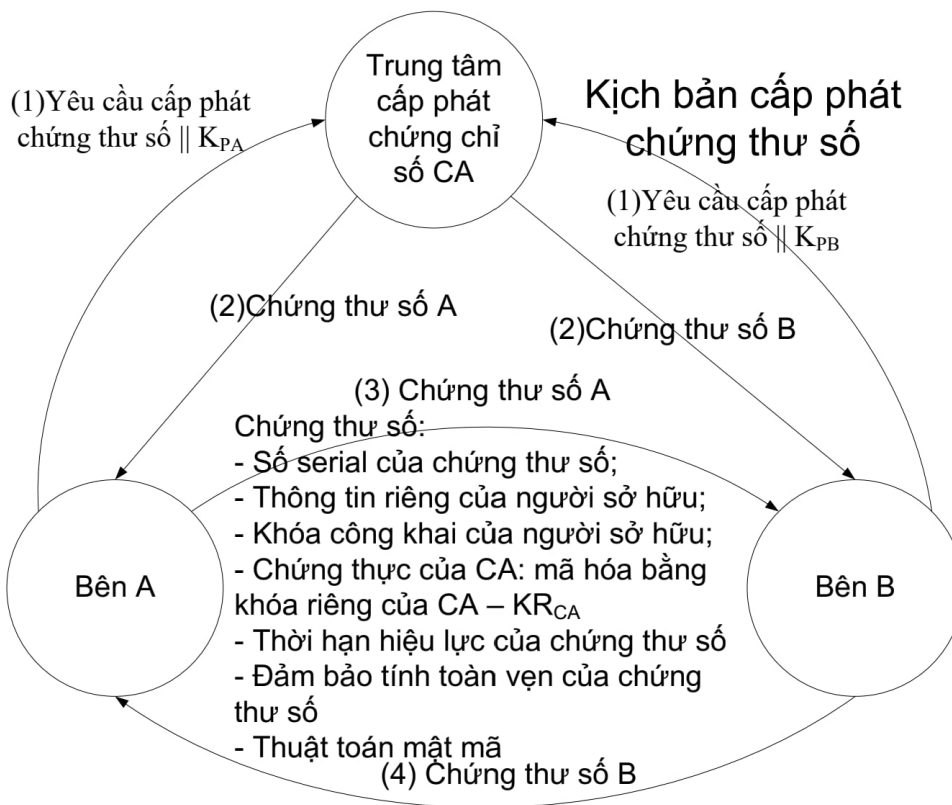


Figure 1.2: Vai trò của CA trong việc ràng buộc danh tính máy chủ với khóa công khai

1.2 Phạm vi và bố cục báo cáo

Báo cáo này tập trung nghiên cứu về Hạ tầng khóa công khai (Public Key Infrastructure – PKI), với mục tiêu làm rõ vai trò của PKI trong việc đảm bảo an toàn thông tin trong

các hệ thống hiện đại. Nội dung báo cáo tiếp cận từ góc độ hệ thống và ứng dụng, không đi sâu vào các chứng minh toán học phức tạp của các thuật toán mật mã, mà tập trung vào việc giải thích nguyên lý hoạt động, kiến trúc và cách thức triển khai trong thực tế.

Cách tiếp cận của báo cáo được tổ chức theo hướng mỗi chương trả lời một câu hỏi cụ thể, giúp người đọc từng bước xây dựng hiểu biết một cách logic và có hệ thống.

Chương 2 trả lời câu hỏi “Tại sao cần PKI?” thông qua việc trình bày các nền tảng của mật mã đối xứng và bất đối xứng, cũng như các hạn chế còn tồn tại của chúng.

Chương 3 tập trung vào câu hỏi “PKI gồm những thành phần nào?”, trong đó mô tả kiến trúc hệ thống và các thực thể như Certification Authority (CA), Registration Authority (RA) và các cơ chế quản lý chứng chỉ.

Chương 4 trả lời câu hỏi “Chứng chỉ số được định nghĩa như thế nào?”, trình bày cấu trúc và vai trò của chứng chỉ số X.509 trong việc ràng buộc giữa danh tính và khóa công khai.

Chương 5 làm rõ “PKI được sử dụng ở đâu trong thực tế?”, với các ví dụ tiêu biểu như HTTPS, chữ ký số và các hệ thống bảo mật khác.

Chương 6 khảo sát các công cụ và nền tảng mã nguồn mở cho phép triển khai PKI trong thực tế, trong khi Chương 7 thảo luận về các thách thức hiện tại cũng như xu hướng phát triển trong tương lai của PKI.

Chapter 2

Cơ sở Mật mã học và Sự tiến hóa của Hệ thống Khóa

Trong kỷ nguyên số, an toàn thông tin được xây dựng trên những nền tảng toán học phức tạp của Mật mã học (Cryptography). Tuy nhiên, lịch sử phát triển của ngành mật mã không phải là một đường thẳng, mà là một quá trình tiến hóa liên tục để giải quyết những nghịch lý phát sinh từ chính các giải pháp trước đó. Sự tiến hóa này bắt nguồn từ mật mã đối xứng với tốc độ vượt trội nhưng vướng mắc ở khâu phân phối khóa; dẫn đến sự ra đời của mật mã bất đối xứng nhằm giải quyết bài toán trao đổi khóa nhưng lại tạo ra lỗ hổng về xác thực danh tính. Cuối cùng, Hạ tầng Khóa công khai (PKI) xuất hiện như một mảnh ghép hoàn thiện, kết nối sức mạnh toán học với cơ chế quản lý niềm tin trong thế giới thực. Chương này sẽ trình bày chi tiết chuỗi logic nền tảng đó.

2.1 Mật mã đối xứng và hạn chế

Nguyên lý hoạt động của Mật mã đối xứng

Mật mã đối xứng (Symmetric-key Cryptography), hay còn gọi là mật mã khóa bí mật, là hình thức mã hóa cổ điển và lâu đời nhất trong lịch sử nhân loại. Nguyên lý cốt lõi của hệ thống này nằm ở tính "đối xứng" của chìa khóa: người gửi và người nhận sử dụng **cùng một khóa duy nhất (K)** cho cả hai quá trình mã hóa (Encryption) và giải mã (Decryption).

Về mặt toán học, quá trình này có thể được biểu diễn thông qua hai hàm cơ bản:

$$C = E(K, P)$$

$$P = D(K, C)$$

Trong đó, P là bản rõ (Plaintext), C là bản mã (Ciphertext), hàm E và D lần lượt là thuật toán mã hóa và giải mã sử dụng khóa bí mật K .

Ưu điểm tuyệt đối và lý do khiến mật mã đối xứng vẫn là trụ cột không thể thay thế trong truyền tải dữ liệu hiện đại chính là **tốc độ xử lý (High Performance)**. Các thuật toán đối xứng tiêu biểu như **DES** (Data Encryption Standard), **3DES** (Triple DES) và đặc biệt là **AES** (Advanced Encryption Standard) được thiết kế dựa trên các phép toán mức bit cực kỳ đơn giản đối với phần cứng máy tính (như phép XOR, dịch chuyển bit Shift, và hoán vị Permutation). Thay vì phải giải quyết các bài toán toán học phức tạp như hệ mật mã bất đối xứng, AES xử lý dữ liệu theo từng khối (Block cipher) với tốc độ có thể lên tới hàng Gigabytes trên giây, đáp ứng hoàn hảo cho việc truyền phát Video, mã hóa ổ cứng hay các luồng dữ liệu thời gian thực.

Bài toán phân phối khóa (The Key Distribution Problem)

Mặc dù sở hữu hiệu năng xuất sắc, mật mã đối xứng lại mang trong mình một tử huyệt mang tính hệ thống khi triển khai trên quy mô lớn: **Bài toán phân phối khóa**.

Nghịch lý của hệ thống đối xứng nằm ở một vòng luẩn quẩn (Catch-22): Để hai bên Alice và Bob có thể giao tiếp an toàn qua mạng Internet, họ cần phải chia sẻ trước với nhau một chiếc khóa K . Tuy nhiên, để gửi chiếc khóa K này cho nhau một cách an toàn mà không bị kẻ thứ ba nghe lén, họ lại cần một kênh liên lạc đã được bảo mật từ trước. Nếu họ đã có sẵn một kênh bảo mật, thì họ đâu cần thiết lập hệ thống mật mã làm gì nữa? Trong quá khứ, giải pháp duy nhất là trao đổi khóa qua một kênh ngoại tuyến (out-of-band channel) như gặp mặt trực tiếp hoặc gửi qua người đưa thư (courier), điều này là hoàn toàn bất khả thi trong kỷ nguyên thương mại điện tử toàn cầu.

Hơn thế nữa, bài toán phân phối khóa tạo ra một rào cản khủng khiếp về mặt mở rộng quy mô (Scalability). Trong một mạng lưới có n thực thể (node), nếu mỗi cặp thực thể đều muốn liên lạc bảo mật với nhau, hệ thống sẽ yêu cầu số lượng khóa phải quản lý tăng theo cấp số nhân. Công thức tính tổng số cặp khóa riêng biệt trong mạng là:

$$\text{Tổng số khóa} = \frac{n(n-1)}{2} = O(n^2)$$

Hãy thử làm một phép tính thực tế:

- Nếu mạng nội bộ chỉ có 10 nhân viên, số khóa cần quản lý là $10 \times 9/2 = 45$ khóa. (Có thể quản lý thủ công).
- Nếu một hệ thống ngân hàng có 1,000 khách hàng, số khóa cần tạo ra là gần 500,000 khóa.
- Nếu triển khai trên phạm vi Internet với 1 triệu người dùng, chúng ta sẽ cần tới xấp xỉ **500 tỷ** chiếc khóa bí mật khác nhau.

Việc lưu trữ, phân phối, cập nhật và thu hồi hàng tỷ chiếc khóa bí mật này tạo ra một cơn ác mộng về mặt quản trị mạng, khiến mô hình đối xứng thuần túy trở nên hoàn toàn

vô dụng đối với môi trường mạng phi tập trung.

Giải pháp KDC (Key Distribution Center) và những giới hạn chí mạng

Để khắc phục bài toán $O(n^2)$ và sự chồng kênh trong việc quản lý khóa, các nhà khoa học máy tính đã đề xuất mô hình **Trung tâm Phân phối Khóa (Key Distribution Center - KDC)**.

Thay vì mỗi người dùng phải ghi nhớ hàng vạn chiếc khóa để nói chuyện với hàng vạn người khác, KDC đóng vai trò là một Bên thứ ba đáng tin cậy (Trusted Third Party). Cơ chế hoạt động của hệ thống được tinh giản như sau:

1. Mỗi người dùng trên mạng chỉ cần chia sẻ **một chiếc khóa chủ (Master Key) duy nhất** với KDC. Lúc này số lượng khóa hệ thống phải quản lý giảm từ $O(n^2)$ xuống chỉ còn $O(n)$.
2. Khi Alice muốn liên lạc với Bob, Alice không liên hệ trực tiếp với Bob mà gửi yêu cầu lên KDC.
3. KDC sẽ sinh ra một chiếc **Khóa phiên ngẫu nhiên (Session Key)**. KDC mã hóa chiếc khóa phiên này bằng khóa chủ của Alice rồi gửi cho Alice; đồng thời mã hóa chiếc khóa phiên đó bằng khóa chủ của Bob rồi gửi cho Bob.
4. Alice và Bob giải mã để lấy Khóa phiên, và sử dụng nó để nhắn tin trực tiếp với nhau. Khi phiên liên lạc kết thúc, chiếc khóa phiên này sẽ bị hủy bỏ.

Giao thức nổi tiếng nhất triển khai mô hình KDC này chính là **Kerberos** [1], được sử dụng rộng rãi trong môi trường mạng nội bộ của doanh nghiệp (như Windows Active Directory). Nó giải quyết xuất sắc bài toán mở rộng quy mô.

Tuy nhiên, KDC không phải là viên đạn bạc. Nó mang trong mình những **giới hạn chí mạng** khiến nó không thể trở thành chuẩn mực cho môi trường Internet mở:

- **Điểm lỗi tập trung (Single Point of Failure - SPOF):** KDC là trái tim của toàn bộ mạng lưới. Nếu máy chủ KDC bị sập (do lỗi phần cứng, mất điện, hoặc bị tấn công DDoS), không một ai trong hệ thống có thể thiết lập được phiên liên lạc mới. Toàn bộ mạng lưới sẽ bị tê liệt hoàn toàn.
- **Nút thắt cổ chai hiệu năng (Performance Bottleneck):** Mọi yêu cầu liên lạc mới của hàng triệu người dùng đều phải dội ngược về KDC để xin cấp Session Key. Điều này đòi hỏi KDC phải có năng lực xử lý khổng lồ, rất dễ gây ra tắc nghẽn mạng (Network congestion).
- **Thảm họa bảo mật (God-mode vulnerability):** KDC là "chiếc rương" chứa mọi khóa chủ của tất cả người dùng. Nếu một nhóm hacker (hoặc nội gián) xâm

nhập thành công vào máy chủ KDC, chúng có khả năng nghe lén, giải mã, hoặc giả mạo bất kỳ thông điệp nào của bất kỳ ai trong hệ thống. Cả mạng lưới sẽ sụp đổ về mặt an toàn thông tin.

- **Vấn đề về ranh giới niềm tin (Trust Boundary):** KDC yêu cầu mọi thành viên phải tin tưởng tuyệt đối vào cơ quan quản lý KDC. Điều này hoạt động tốt trong một công ty (nhân viên tin tưởng phòng IT). Nhưng trên môi trường Internet mở, nơi Alice ở Việt Nam muốn mua hàng của Bob ở Mỹ, không tồn tại một tổ chức KDC chung nào mà cả hai quốc gia và cả hai cá nhân đều sẵn sàng giao phó toàn bộ khóa bí mật của mình.

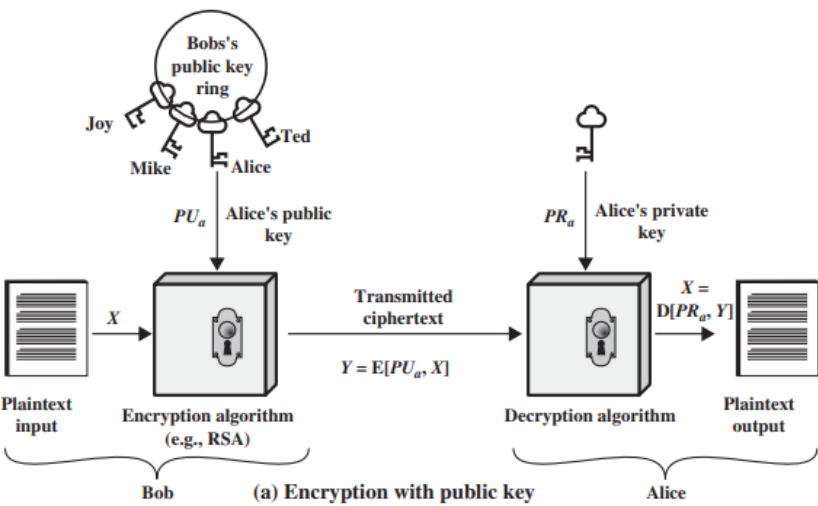
Sự bế tắc của KDC trong việc thiết lập niềm tin trên diện rộng đã đặt ra một yêu cầu bức thiết cho ngành Mật mã học: Cần phải có một hệ thống cho phép hai người hoàn toàn xa lạ giao tiếp bảo mật với nhau mà không cần phải chia sẻ trước bất kỳ bí mật nào, cũng không cần phải trao quyền sinh sát cho một trung tâm KDC tập trung. Và đó chính là lúc Mật mã khóa bất đối xứng (Asymmetric Cryptography) bước ra ánh sáng.

2.2 Mật mã khóa công khai

Cặp khóa bất đối xứng và Trapdoor function

Khác với hệ mật mã đối xứng chỉ sử dụng một khóa duy nhất, mật mã khóa công khai (Public-key Cryptography) hoạt động dựa trên một cặp khóa gắn kết chặt chẽ về mặt toán học: **Khóa công khai (Public Key - PU)** và **Khóa bí mật (Private Key - PR)**. Tùy thuộc vào cách phối hợp và thứ tự sử dụng hai chiếc khóa này, hệ thống có thể giải quyết ba bài toán cốt lõi của an toàn thông tin:

1. **Đảm bảo tính bảo mật (Confidentiality):** Người gửi mã hóa thông điệp bằng *PU* của người nhận. Lúc này, bản mã chỉ có thể được giải mã bằng *PR* duy nhất mà người nhận đang cất giữ. Kịch bản này bảo vệ dữ liệu khỏi những kẻ nghe lén trên đường truyền.
2. **Xác thực và Chữ ký số (Authentication/Digital Signature):** Người gửi "mã hóa" (hoặc ký) thông điệp bằng chính *PR* của mình. Bất kỳ ai trên mạng cũng có thể dùng *PU* của người gửi để giải mã. Nếu quá trình giải mã thành công, điều này minh chứng tuyệt đối rằng thông điệp thực sự xuất phát từ người gửi đó và nội dung không hề bị chỉnh sửa (chống chối cãi).
3. **Bảo mật và Xác thực song hành:** Người gửi kết hợp cả hai thao tác. Đầu tiên, ký thông điệp bằng *PR* của nguồn gửi để xác thực, sau đó tiếp tục bọc thêm một lớp mã hóa bằng *PU* của đích đến để bảo mật. Nhờ vậy, thông điệp vừa được giữ kín, vừa đảm bảo tính chính danh.



(a) Mã hóa bằng khóa công khai của người nhận

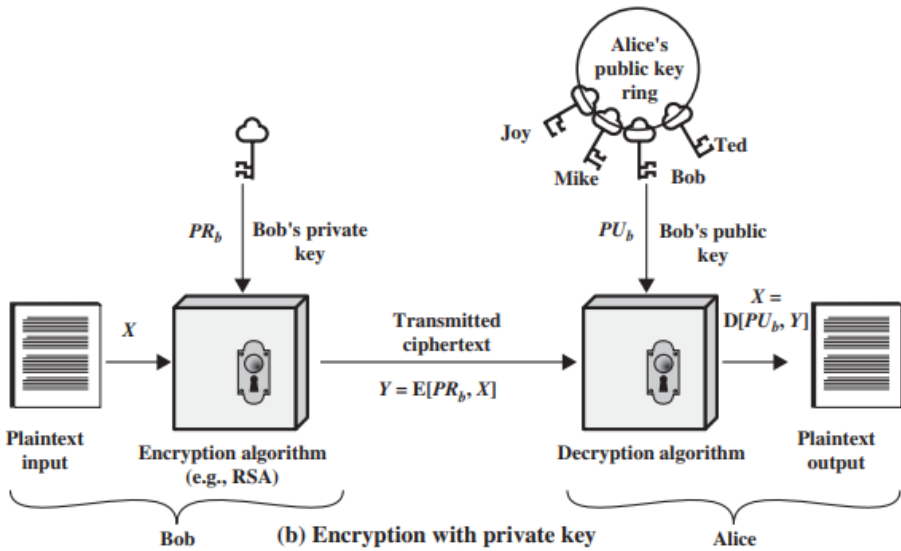
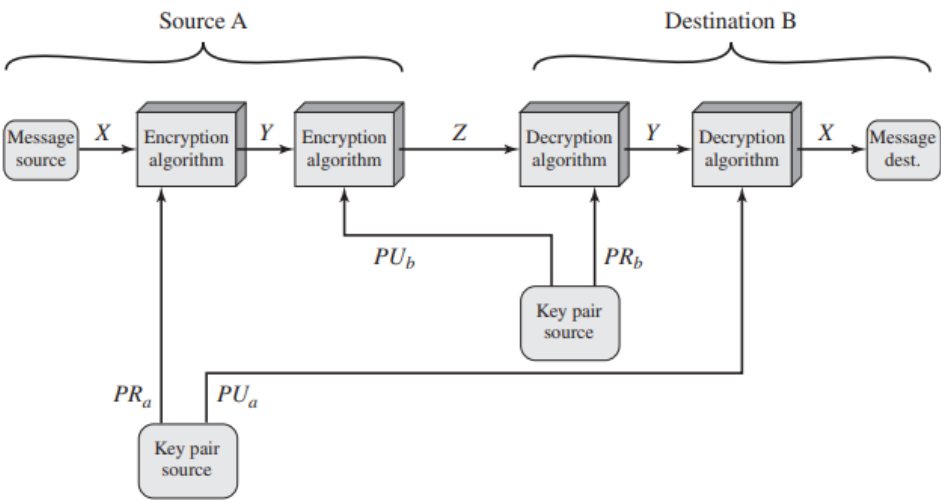


Figure 9.1 Public-Key Cryptography

(b) Xác thực / chữ ký bằng khóa riêng người gửi



(c) Bảo mật và Xác thực kết hợp

Figure 2.1: Ba kịch bản ứng dụng cốt lõi của hệ mật mã khóa bất đối xứng

Kịch bản 1: Đảm bảo tính bảo mật (Encryption with Public Key) Mục tiêu của kịch bản này là đảm bảo chỉ duy nhất người nhận hợp pháp mới có thể đọc được nội dung thông điệp (Confidentiality).

- **Phía người gửi (Bob):** Bob lấy thông điệp gốc X và tra cứu Khóa công khai của người nhận (Alice) trên một vòng khóa (public key ring). Sau đó, Bob thực hiện mã hóa:

$$Y = E[PU_a, X]$$

Bản mã Y lúc này được truyền đi trên kênh không an toàn.

- **Phía người nhận (Alice):** Khi nhận được Y , Alice sử dụng Khóa bí mật duy nhất của mình (PR_a) để giải mã:

$$X = D[PR_a, Y]$$

Vì chỉ có Alice mới nắm giữ PR_a , mọi kẻ nghe lén dù bắt được Y và biết PU_a cũng không thể đảo ngược quá trình để tìm ra X .

Kịch bản 2: Xác thực nguồn gốc và Chữ ký số (Encryption with Private Key) Mục tiêu ở đây không phải là giấu kín dữ liệu, mà là chứng minh tuyệt đối nguồn gốc của thông điệp (Authentication).

- **Phía người gửi (Bob):** Bob dùng chính Khóa bí mật của mình (PR_b) để "mã hóa" (bản chất là ký số) lên thông điệp X :

$$Y = E[PR_b, X]$$

- **Phía người nhận (Alice):** Alice nhận Y và tra cứu Khóa công khai của Bob (PU_b) để khôi phục:

$$X = D[PU_b, Y]$$

Bất kỳ ai trên mạng cũng có thể dùng PU_b để đọc được thông điệp này. Tuy nhiên, nếu hàm giải mã D chạy thành công và trả về dữ liệu có nghĩa, điều đó minh chứng 100% rằng bản mã Y *bắt buộc* phải được tạo ra từ PR_b của Bob. Bob không thể chối cãi việc mình đã gửi thông điệp này.

Kịch bản 3: Bảo mật và Xác thực song hành (Secrecy and Authentication) Để đáp ứng cả hai yêu cầu khắt khe nhất trong liên lạc, hệ thống sẽ bọc dữ liệu qua hai lớp vỏ mã hóa, kết hợp cả hai kịch bản trên.

- **Phía người gửi (Source A):** Đầu tiên, A tạo chữ ký để xác thực bằng cách mã hóa với Khóa bí mật của mình: $Y = E[PR_a, X]$. Sau đó, A bọc lớp bảo mật bên

ngoài bằng cách mã hóa tiếp Y với Khóa công khai của đích đến B:

$$Z = E[PU_b, Y]$$

- **Phía người nhận (Destination B):** Quá trình bóc tách diễn ra ngược lại. B dùng Khóa bí mật của mình để phá lớp vỏ bảo mật: $Y = D[PR_b, Z]$. Cuối cùng, B dùng Khóa công khai của A để xác thực và lấy ra thông điệp gốc:

$$X = D[PU_a, Y]$$

Nền tảng toán học: Hàm một chiều có cửa sập (Trapdoor One-way Function)

Để các kịch bản trên có thể hoạt động an toàn trong thực tế, hệ thống đòi hỏi một cơ chế toán học cực kỳ khắt khe. Việc công khai PU và thuật toán mã hóa cho cả thế giới biết đặt ra thách thức: Làm sao để ngăn chặn hacker tính ngược lại để tìm ra PR hoặc bản rõ ban đầu? Câu trả lời nằm ở khái niệm **Hàm một chiều có cửa sập**.

Khái niệm này bao gồm hai tính chất then chốt:

- **Hàm một chiều (One-way Function):** Là một hàm số $Y = f(X)$ thỏa mãn điều kiện: nếu biết X , việc tính xuôi ra Y là rất dễ dàng (thời gian đa thức). Tuy nhiên, nếu chỉ biết Y , việc tính ngược $X = f^{-1}(Y)$ là *bất khả thi về mặt tính toán* (*computationally infeasible*) đối với máy tính hiện hành.
- **Cửa sập (Trapdoor):** Hệ mật mã không thể chỉ dùng hàm một chiều đơn thuần (vì chính người nhận cũng sẽ không giải mã được). Do đó, hàm này được thiết kế thêm một "cửa sập". Nếu sở hữu thông tin bổ sung k (chính là Khóa bí mật PR), người nhận có thể kích hoạt cửa sập và dễ dàng tính toán ngược $X = f_k^{-1}(Y)$.

Tóm lại, sự sinh tồn của mật mã bất đối xứng phụ thuộc vào việc xây dựng thành công hàm tính toán một chiều vững chắc, nơi Khóa công khai đóng vai trò là hàm f để chốt khóa, còn Khóa bí mật chính là chìa khóa cửa sập k để mở luồng tính ngược.

Thuật toán RSA

Thuật toán RSA, được đặt theo tên của ba nhà phát minh Rivest, Shamir và Adleman, là một trong những hệ mật mã khóa công khai phổ biến và nền tảng nhất. Tính an toàn của RSA dựa trên độ khó của **bài toán phân tích thừa số nguyên tố lớn (Integer Factorization Problem)** [2]. Về mặt toán học, việc nhân hai số nguyên tố khổng lồ để tạo ra một hợp số thì diễn ra rất nhanh, nhưng việc làm ngược lại — phân tích hợp số đó ra hai thừa số ban đầu — là một bài toán bất khả thi (infeasible) đối với khả năng điện toán của máy tính hiện tại.

Quy trình hoạt động của RSA Hệ thống RSA được vận hành qua 3 giai đoạn cốt lõi:

1. Tạo khóa (Key Generation): (Thực hiện bởi người nhận - Alice)

- Chọn hai số nguyên tố lớn ngẫu nhiên p và q ($p \neq q$).
- Tính modulus $n = p \times q$.
- Tính giá trị hàm Euler totient: $\phi(n) = (p-1)(q-1)$.
- Chọn một số nguyên e sao cho $1 < e < \phi(n)$ và $\gcd(\phi(n), e) = 1$ (tức là e và $\phi(n)$ nguyên tố cùng nhau).
- Tính khóa giải mã d là nghịch đảo của e theo modulo $\phi(n)$, tức là:

$$d \equiv e^{-1} \pmod{\phi(n)}$$

- **Khóa công khai (Public Key):** $PU = \{e, n\}$.
- **Khóa bí mật (Private Key):** $PR = \{d, n\}$.
(Lưu ý: Các giá trị p, q và $\phi(n)$ phải được tiêu hủy hoặc cất giữ tuyệt mật sau khi tạo khóa xong).

2. Mã hóa (Encryption): (Thực hiện bởi người gửi - Bob)

- Bob muốn gửi thông điệp M cho Alice (với điều kiện $M < n$).
- Bob sử dụng Khóa công khai PU của Alice để tính bản mã C :

$$C = M^e \bmod n$$

3. Giải mã (Decryption): (Thực hiện bởi người nhận - Alice)

- Alice nhận bản mã C và sử dụng Khóa bí mật PR của mình để khôi phục nguyên trạng thông điệp M :

$$M = C^d \bmod n$$

RSA.png RSA.png

Figure 2.2: Sơ đồ tạo khóa, mã hóa và giải mã của thuật toán RSA (Nguồn: Figure 9.5 & 9.6)

Tính bảo mật của RSA Độ an toàn của hệ mật mã RSA phải đối mặt với 4 hướng tấn công chủ đạo từ giới phân tích mật mã (cryptanalysis):

- 1. Tấn công vét cạn (Brute force):** Thử tất cả các khóa bí mật (Private Key) có thể. Hướng này bị vô hiệu hóa bằng cách sử dụng không gian khóa khổng lồ.

2. **Tấn công toán học (Mathematical attacks):** Tập trung vào việc giải bài toán phân tích thừa số nguyên tố lớn để dịch ngược các thông số cốt lõi.
3. **Tấn công thời gian (Timing attacks):** Một dạng tấn công kênh kề (side-channel), đo lường thời gian chạy của thuật toán giải mã để suy luận ra từng bit của khóa bí mật.
4. **Tấn công bản mã lựa chọn (Chosen Ciphertext attacks - CCA):** Khai thác các tính chất toán học cơ bản của thuật toán RSA để tạo ra các thông điệp có chủ đích.

1. Bài toán phân tích thừa số (The Factoring Problem)

Phòng tuyến toán học đầu tiên và quan trọng nhất của RSA là khó khăn trong việc phân tích một hợp số lớn n thành hai thừa số nguyên tố p và q . Nếu kẻ tấn công có thể phân tích thành công n , chúng sẽ dễ dàng tính được $\phi(n) = (p-1)(q-1)$ và từ đó trích xuất được khóa giải mã d . Mặc dù các thuật toán phân tích (như General Number Field Sieve - GNFS) ngày càng tiến bộ, nhưng nỗ lực tính toán vẫn tăng theo cấp số mũ so với độ dài của khóa. Do đó, việc nâng kích thước khóa RSA lên tiêu chuẩn 2048-bit hoặc 4096-bit được coi là an toàn tuyệt đối trước các đòn tấn công toán học truyền thống.

2. Tấn công thời gian (Timing Attack) và Cách khắc phục

Timing Attack là một kịch bản tấn công thực tế và cực kỳ nguy hiểm do Paul Kocher đề xuất. Kẻ tấn công không cần giải toán mà sẽ đo lường thời gian phản hồi (chậm/nhanh) của bộ xử lý khi thực thi thuật toán lũy thừa modulo (Square and Multiply). Sự chênh lệch thời gian xử lý giữa bit '0' và bit '1' của khóa d sẽ tố cáo toàn bộ giá trị của khóa.

Để chống lại đòn tấn công này, có 3 biện pháp kỹ thuật được áp dụng:

- **Cố định thời gian tính toán (Constant exponentiation time):** Buộc hệ thống luôn phải tiêu tốn một khoảng thời gian cố định cho mọi phép tính, bất kể bit đó là '0' hay '1'. Tuy nhiên, điều này làm giảm hiệu năng hệ thống.
- **Thêm độ trễ ngẫu nhiên (Random delay):** Chèn thêm các khoảng thời gian chờ ngẫu nhiên vào thuật toán để làm nhiễu thông số đo lường của kẻ tấn công.
- **Tung hỏa mù (Blinding):** Đây là biện pháp tối ưu nhất. Trước khi giải mã, bản mã C được nhân với một giá trị ngẫu nhiên r : $C' = C \cdot r^e \bmod n$. Bộ xử lý sẽ giải mã trên một cục dữ liệu rác C' , khiến mọi phép đo thời gian của hacker trở nên vô nghĩa, sau đó hệ thống sẽ khử r ở bước cuối cùng để trả về bản rõ M .

3. Tấn công CCA và chuẩn OAEP (Optimal Asymmetric Encryption Padding)

Thuật toán RSA nguyên thủy (Basic RSA) có một lỗ hổng nghiêm trọng dựa trên đặc

tính nhân (multiplicative property): Mã hóa của M_1 nhân với mã hóa của M_2 sẽ bằng mã hóa của $(M_1 \times M_2)$. Kẻ tấn công có thể lợi dụng điều này bằng cách chèn thêm các hệ số nhân vào bản mã C , biến đổi nó thành một bản mã có chủ đích (Chosen Ciphertext) và lừa máy chủ giải mã để lấy thông tin.

Để vô hiệu hóa hoàn toàn các phương pháp tấn công này, hệ thống RSA thực tế không bao giờ mã hóa trực tiếp thông điệp M , mà phải đi qua một cơ chế "đệm" cực kỳ phức tạp có tên là **OAEP (Optimal Asymmetric Encryption Padding)**.

Về bản chất, OAEP sẽ trộn lẫn thông điệp M ban đầu với các chuỗi padding, các giá trị băm $H(P)$ và một hạt giống ngẫu nhiên (Seed). Thông qua mạng lưới kết hợp các hàm sinh mặt nạ (Mask Generating Function - MGF) và phép toán XOR chéo, OAEP sẽ nhào nặn thông điệp thành một khối dữ liệu (Encoded Message - EM) hoàn toàn hỗn độn và không còn bất kỳ mối liên hệ toán học tuyến tính nào. Nếu kẻ tấn công cố tình nhân thêm hệ số vào bản mã, khi giải mã, cấu trúc OAEP sẽ bị phá vỡ, máy chủ sẽ phát hiện lỗi và từ chối trả về kết quả.

OAEP.pngOAEP.png

Figure 2.3: Sơ đồ khối quá trình nhào nặn dữ liệu của chuẩn OAEP (Nguồn: Figure 9.10)

Trao đổi khóa Diffie-Hellman

Diffie-Hellman (DH) là một trong những hệ thống mật mã khóa công khai (Public-key Cryptography) đầu tiên được công bố. Khác với tính đa dụng của RSA, thuật toán DH được thiết kế với một mục đích duy nhất và chuyên biệt: **Trao đổi khóa (Key Exchange)**. Sứ mệnh cốt lõi của DH là cho phép hai thực thể hoàn toàn xa lạ có thể đàm phán và thiết lập một Khóa bí mật chung (Shared Secret / Session Key) một cách an toàn thông qua một kênh truyền tải không bảo mật, nơi mọi thông điệp đều có thể bị kẻ tấn công nghe lén. Khóa chung này sau đó sẽ được sử dụng làm đầu vào cho các thuật toán mã hóa đối xứng (như AES) nhằm truyền tải dữ liệu ở tốc độ cao.

Về mặt toán học, độ an toàn của thuật toán Diffie-Hellman phụ thuộc hoàn toàn vào độ khó của **Bài toán Logarit rời rạc (Discrete Logarithm Problem)**. Thuật toán sử dụng số nguyên tố p rất lớn và một căn nguyên thủy (primitive root) a của nó. Trong không gian số học modulo p , việc tính toán xuôi $a^i \bmod p$ là một thao tác cực kỳ nhanh chóng. Tuy nhiên, thao tác tính toán ngược — tức là tìm lại số mũ bí mật i khi chỉ biết kết quả b , cơ số a và số modulo p ($i = \log_a b \pmod{p}$) — là một bài toán bất khả thi về mặt thời gian (infeasible) đối với các hệ thống máy tính hiện tại.

Lỗ hổng bảo mật: Mặc dù giao thức DH nguyên thủy vô hiệu hóa hoàn toàn khả năng giải mã của kẻ nghe lén thụ động (Passive Eavesdropping), nó lại tồn tại một điểm yếu chí mạng: Không có cơ chế xác thực danh tính. Do đó, thuật toán này cực kỳ dễ bị khai thác bởi đòn tấn công **Man-in-the-Middle (MitM)**. Để khắc phục trong các ứng dụng thực tế (như giao thức TLS/SSL), Diffie-Hellman bắt buộc phải được sử dụng kết

hợp với Chữ ký số (Digital Signatures) và Chứng chỉ số (Digital Certificates) để đảm bảo tính xác thực của các bên tham gia.

Các bước thực hiện thuật toán Diffie-Hellman Giả sử hai người dùng A (Alice) và B (Bob) muốn thiết lập một khóa bí mật chung qua một kênh truyền không an toàn. Thuật toán được thực hiện qua 3 bước cốt lõi:

1. Khởi tạo thông số công khai (Global Public Elements):

Trước khi trao đổi, Alice và Bob phải thống nhất hai thông số nền tảng (các giá trị này có thể truyền công khai):

- q : Một số nguyên tố rất lớn.
- α : Một căn nguyên thủy (primitive root) của q , với điều kiện $\alpha < q$.

2. Tạo khóa cá nhân và khóa công khai (Key Generation):

• **Phía Alice:**

- Chọn một số nguyên ngẫu nhiên bí mật $X_A < q$ (đóng vai trò là Private Key).
- Tính giá trị trộn công khai $Y_A = \alpha^{X_A} \bmod q$ và gửi Y_A cho Bob.

• **Phía Bob:**

- Tương tự, chọn một số nguyên ngẫu nhiên bí mật $X_B < q$.
- Tính giá trị trộn công khai $Y_B = \alpha^{X_B} \bmod q$ và gửi Y_B cho Alice.

3. Tính toán khóa chung bí mật (Calculation of Secret Key):

Sau khi nhận được giá trị Y từ đối phương, hai bên tiến hành tính toán khóa phiên (Session Key) K độc lập tại hệ thống của mình:

- Alice tính: $K = (Y_B)^{X_A} \bmod q$
- Bob tính: $K = (Y_A)^{X_B} \bmod q$

Chứng minh tính đồng nhất của khóa K Dựa vào các tính chất cơ bản của số học modulo, hai biểu thức tính toán độc lập trên thực chất đều hội tụ về một kết quả duy nhất:

$$K = (Y_B)^{X_A} \bmod q = (\alpha^{X_B} \bmod q)^{X_A} \bmod q = \alpha^{X_B \cdot X_A} \bmod q$$

$$K = (Y_A)^{X_B} \bmod q = (\alpha^{X_A} \bmod q)^{X_B} \bmod q = \alpha^{X_A \cdot X_B} \bmod q$$

Do phép nhân số mũ có tính chất giao hoán ($X_A \cdot X_B = X_B \cdot X_A$), cả Alice và Bob đều tổng hợp ra được một chiếc khóa K giống hệt nhau mà không cần phải gửi trực tiếp K qua mạng.

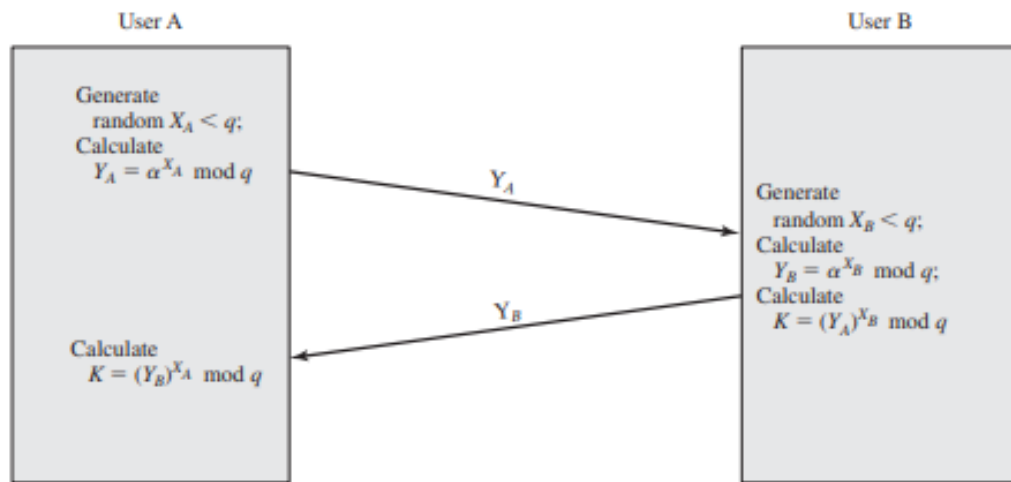


Figure 10.2 Diffie-Hellman Key Exchange

Figure 2.4: Sơ đồ luồng trao đổi khóa Diffie-Hellman (Nguồn: Figure 10.2)

Phân tích độ an toàn trước kẻ tấn công Giả sử có một kẻ nghe lén (Eavesdropper) kiểm soát toàn bộ kênh truyền, kẻ này sẽ thu thập được đầy đủ 4 thông số công khai: q, α, Y_A và Y_B . Tuy nhiên, để tìm ra được khóa chung K , kẻ tấn công bắt buộc phải trích xuất được số mũ bí mật X_A hoặc X_B . Quá trình này đòi hỏi kẻ tấn công phải giải phương trình:

$$X_B = \text{dlog}_{\alpha, q}(Y_B)$$

Trong đó dlog là phép tính logarit rời rạc. Nếu tham số q được chọn đủ lớn (theo tiêu chuẩn hiện tại là từ 2048-bit trở lên), sức mạnh điện toán hiện tại của nhân loại là không thể giải quyết bài toán tính ngược này trong thời gian khả thi (computationally infeasible).

Điểm yếu chí mạng: Tấn công Man-in-the-Middle (MitM) Mặc dù giao thức Diffie-Hellman an toàn tuyệt đối trước những kẻ nghe lén thụ động, nó lại hoàn toàn bất lực trước đòn tấn công chủ động can thiệp vào đường truyền, điển hình là **Tấn công Kẻ đứng giữa (Man-in-the-Middle - MitM)**. Lỗ hổng cốt lõi xuất phát từ việc thuật toán không có cơ chế xác thực danh tính (Authentication) của các bên tham gia.

Kịch bản tấn công MitM:

Giả sử kẻ tấn công (Darth) kiểm soát được đường truyền giữa Alice và Bob. Quá trình đánh chặn diễn ra như sau:

1. **Khởi tạo khóa giả mạo:** Darth tự tạo ra hai bộ khóa bí mật ngẫu nhiên X_{D1} và X_{D2} , từ đó tính ra hai giá trị công khai giả mạo tương ứng là Y_{D1} và Y_{D2} .
2. **Đánh chặn và tráo đổi:**
 - Khi Alice gửi Y_A cho Bob, Darth đánh chặn Y_A , tự động gửi Y_{D1} cho Bob. Lúc

này Darth tính được khóa chung thứ nhất: $K2 = (Y_A)^{X_{D2}} \bmod q$.

- Khi Bob gửi Y_B cho Alice, Darth tiếp tục đánh chặn Y_B , gửi Y_{D2} cho Alice. Darth tính được khóa chung thứ hai: $K1 = (Y_B)^{X_{D1}} \bmod q$.

3. Ảo giác hệ thống:

- Bob nhận Y_{D1} (tưởng của Alice) và tính ra khóa $K1$. Thực chất Bob đang chia sẻ khóa $K1$ với Darth.
- Alice nhận Y_{D2} (tưởng của Bob) và tính ra khóa $K2$. Thực chất Alice đang chia sẻ khóa $K2$ với Darth.

4. **Thao túng dữ liệu:** Mọi luồng liên lạc trong tương lai đều bị Darth kiểm soát. Nếu Alice gửi thông điệp M mã hóa bằng $K2$, Darth có thể dùng $K2$ để giải mã, đọc trộm hoặc chỉnh sửa thành M' , sau đó mã hóa lại bằng $K1$ và gửi cho Bob. Cả Alice và Bob đều không thể phát hiện ra sự can thiệp này.

Giải pháp khắc phục:

Để giao thức Diffie-Hellman có thể hoạt động an toàn trên thực tế (ví dụ: trong giao thức SSL/TLS), nó bắt buộc phải được kết hợp với các cơ chế định danh như **Chữ ký số (Digital Signatures)** và **Chứng chỉ số khóa công khai (Public-key Certificates)** để đảm bảo các gói tin Y_A và Y_B không bị đánh tráo giữa chừng.

Sơ đồ chữ ký số (Digital Signatures)

Trong các hệ thống xác thực sử dụng khóa đối xứng (Symmetric Authentication), việc trao đổi thông điệp giữa hai bên Alice và Bob luôn tiềm ẩn những nguy cơ tranh chấp pháp lý mà các kỹ thuật mã hóa thông thường không thể giải quyết triệt để. Để hiểu rõ tại sao chúng ta cần đến Chữ ký số, hãy xem xét hai kịch bản rủi ro sau đây: **1. Kịch bản giả mạo nội dung (Forgery by Receiver):**

Giả sử Alice và Bob chia sẻ một khóa bí mật chung để xác thực các giao dịch chuyển tiền. Bob (người nhận) có thể tự ý chỉnh sửa con số trong thông điệp từ 100 USD thành 1000 USD, sau đó dùng khóa chung đó để tạo lại mã xác thực mới. Khi có tranh chấp, Bob có thể khẳng định trước tòa rằng Alice đã gửi 1000 USD, và vì mã xác thực là hợp lệ, Alice không có bằng chứng nào để minh oan.

2. Kịch bản chối bỏ trách nhiệm (Denial of Sending):

Alice gửi một yêu cầu đặt lệnh mua cổ phiếu cho Bob. Tuy nhiên, sau đó thị trường lao dốc khiến Alice thua lỗ. Lúc này, Alice hoàn toàn có thể tuyên bố rằng mình chưa từng gửi thông điệp đó. Vì Bob cũng nắm giữ khóa bí mật chung, Alice có thể đổ lỗi rằng Bob đã tự tạo ra thông điệp để hãm hại mình.

digitroll.pngdigitroll.png

Figure 2.5: Mô hình tổng quát của quá trình ký số và các kịch bản tranh chấp phát sinh (Nguồn: Figure 13.1)

Yêu cầu đối với một Chữ ký số tiêu chuẩn Từ những rủi ro trên, chúng ta rút ra kết luận rằng trong các tình huống mà sự tin tưởng giữa người gửi và người nhận không phải là tuyệt đối, chúng ta cần một cơ chế mạnh mẽ hơn xác thực thông thường. Một sơ đồ chữ ký số (Digital Signature) đúng nghĩa phải thỏa mãn các đặc tính sau:

- **Xác thực tác giả và thời điểm:** Phải có khả năng kiểm chứng chính xác ai là người ký và chữ ký đó được tạo ra vào thời gian nào.
- **Xác thực nội dung:** Tại thời điểm ký, nội dung thông điệp phải được "khóa" lại. Mọi sự thay đổi sau đó đều phải làm cho chữ ký trở nên vô hiệu.
- **Khả năng kiểm chứng bởi bên thứ ba:** Đây là đặc tính quan trọng nhất để giải quyết tranh chấp. Một bên thứ ba độc lập (ví dụ: Trọng tài hoặc Tòa án) phải có khả năng kiểm tra tính hợp lệ của chữ ký mà không cần nắm giữ khóa bí mật của người ký.

Như vậy, về mặt chức năng, chữ ký số bao hàm cả chức năng xác thực thông thường nhưng được nâng cấp thêm khả năng **Chống chối cãi (Non-repudiation)**, biến nó thành một công cụ pháp lý quan trọng trong giao dịch điện tử.

Các hình thức tấn công và mức độ giả mạo chữ ký số Dựa trên nghiên cứu của Goldwasser, Micali và Rivest **GOLD88**, các hình thức tấn công vào hệ thống chữ ký số được phân loại theo mức độ nghiêm trọng tăng dần, dựa trên lượng thông tin mà kẻ tấn công (C) có thể thu thập được từ người dùng hợp pháp (A):

- **Tấn công chỉ dựa trên khóa (Key-only attack):** C chỉ biết được khóa công khai của A . Đây là mức độ cơ bản nhất.
- **Tấn công thông điệp đã biết (Known message attack):** C có danh sách một số thông điệp và chữ ký tương ứng của chúng đã được tạo ra trước đó bởi A .
- **Tấn công chọn thông điệp chung (Generic chosen message attack):** C chọn một danh sách các thông điệp trước khi thực hiện tấn công (độc lập với khóa công khai của A) và yêu cầu A ký vào các thông điệp này.
- **Tấn công chọn thông điệp định hướng (Directed chosen message attack):** Tương tự như tấn công chung, nhưng danh sách thông điệp được C chọn sau khi đã biết khóa công khai của A .

- **Tấn công chọn thông điệp thích ứng (Adaptive chosen message attack):**

Đây là hình thức nguy hiểm nhất. C sử dụng A như một "oracle" để yêu cầu ký các thông điệp mới dựa trên kết quả của những cặp thông điệp-chữ ký đã thu thập được trước đó.

Song song với các kiểu tấn công, sự thành công của kẻ tấn công trong việc phá vỡ hệ thống (Breaking a signature scheme) cũng được chia thành các cấp độ từ "sụp đổ hoàn toàn" đến "gây nhiễu":

- **Phá vỡ hoàn toàn (Total break):** Kẻ tấn công tìm ra được khóa bí mật cá nhân của A . Đây là thất bại nghiêm trọng nhất, làm sụp đổ toàn bộ hệ thống.
- **Giả mạo phổ quát (Universal forgery):** C tìm ra một thuật toán ký hiệu quả có khả năng tạo ra chữ ký hợp lệ cho bất kỳ thông điệp ngẫu nhiên nào, tương đương với việc sở hữu khóa bí mật.
- **Giả mạo có chọn lọc (Selective forgery):** C tạo ra được chữ ký hợp lệ cho một thông điệp cụ thể mà C đã nhắm mục tiêu từ trước.
- **Giả mạo tồn tại (Existential forgery):** C tạo ra được ít nhất một cặp thông điệp-chữ ký hợp lệ. Tuy nhiên, C không kiểm soát được nội dung của thông điệp này (thông điệp có thể là một chuỗi vô nghĩa), nên đôi khi nó chỉ dừng lại ở mức độ gây nhiễu cho hệ thống.

Yêu cầu kỹ thuật đối với một hệ thống Chữ ký số an toàn Dựa trên việc phân tích các hình thức tấn công và nguy cơ giả mạo (Attacks and Forgeries), chúng ta có thể thiết lập một bộ quy tắc khắt khe để đảm bảo tính sinh tồn của một sơ đồ chữ ký số trong thực tế. Các yêu cầu này không chỉ nhắm vào tính bảo mật mà còn chú trọng đến hiệu năng vận hành:

- **Sự phụ thuộc vào nội dung (Message Dependency):** Chữ ký số phải là một chuỗi bit được tạo ra dựa trên chính nội dung của thông điệp được ký. Nếu thông điệp thay đổi dù chỉ 1 bit, chữ ký phải thay đổi hoàn toàn. Điều này ngăn chặn đòn tấn công *Existential Forgery* (giả mạo tồn tại).
- **Tính duy nhất của người gửi (Sender Uniqueness):** Chữ ký phải sử dụng thông tin duy nhất và bí mật của người gửi (Private Key). Yêu cầu này là "lá chắn" cốt lõi để chống lại việc giả mạo (forgery) từ phía người nhận và ngăn chặn việc chối bỏ trách nhiệm (denial) từ phía người gửi.
- **Tính khả thi trong tạo lập (Ease of Production):** Việc tạo ra chữ ký số phải tương đối dễ dàng và nhanh chóng đối với người gửi hợp pháp, không gây quá tải cho tài nguyên hệ thống.

- **Tính dễ dàng trong xác thực (Ease of Verification):** Mọi bên thứ ba hoặc người nhận hợp pháp đều phải có khả năng kiểm tra và xác định tính đúng đắn của chữ ký một cách thuận tiện thông qua khóa công khai.
- **Kháng giả mạo về mặt tính toán (Computational Infeasibility):** Đây là yêu cầu quan trọng nhất để đối phó với *Universal* và *Selective Forgery*. Hệ thống phải đảm bảo rằng kẻ tấn công không thể:
 - Tạo ra một thông điệp mới cho một chữ ký số đã tồn tại.
 - Tạo ra một chữ ký giả mạo cho một thông điệp đã cho sẵn.
- **Khả năng lưu trữ (Storage Practicality):** Bản sao của chữ ký số phải có kích thước tối ưu để có thể lưu trữ lâu dài nhằm phục vụ mục đích đối soát hoặc giải quyết tranh chấp pháp lý sau này.

Để thỏa mãn đồng thời các yêu cầu khắt khe này, các hệ thống hiện đại thường sử dụng một **hàm băm an toàn (Secure Hash Function)** kết hợp với các thuật toán mã hóa bất đối xứng. Hàm băm đảm bảo tính phụ thuộc nội dung và hiệu năng tính toán, trong khi mật mã bất đối xứng đảm bảo tính duy nhất và khả năng xác thực bởi bên thứ ba. Tuy nhiên, việc thiết kế chi tiết các giao thức này (như cách chèn thêm padding) cần được thực hiện cực kỳ cẩn trọng để tránh các lỗ hổng tình vi như tấn công thời gian hay tấn công bản mã lựa chọn.

2.3 Tại sao mật mã bất đối xứng chưa đủ?

Mật mã khóa công khai (Asymmetric Cryptography) đã giải quyết được một trong những bài toán nan giải nhất của ngành mật mã học: Phân phối khóa bí mật qua một kênh truyền không an toàn mà không cần hai bên phải gặp mặt trực tiếp. Bằng các đặc tính toán học ưu việt như hàm một chiều có cửa sập (Trapdoor one-way function), hệ thống này cung cấp cả hai tính năng cốt lõi là bảo mật (Confidentiality) và chữ ký số (Digital Signature).

Tuy nhiên, sức mạnh toán học đơn thuần là chưa đủ để xây dựng một hệ thống an toàn trên quy mô toàn cầu. Lỗ hổng chết người của mật mã bất đối xứng không nằm ở các phương trình toán học, mà nằm ở lỗ hổng về "Niềm tin" (Trust) trong quá trình phân phối và định danh khóa. Khi các khóa công khai thuần túy (Raw Public Keys) được truyền tải trên một mạng lưới mở như Internet, chúng trở thành miếng mồi ngon cho các đòn tấn công có chủ đích.

Tấn công Man-in-the-Middle (MITM) trên Raw Public Key

Để minh họa cho điểm yếu này, hãy xem xét kịch bản trao đổi khóa công khai nguyên thủy (Raw Public Key Exchange). Giả sử Alice muốn gửi một tài liệu mật cho Bob. Theo

lý thuyết, Alice chỉ cần xin khóa công khai của Bob ($K_{\text{pub_Bob}}$) để mã hóa tài liệu. Tuy nhiên, trên một môi trường mạng bị kiểm soát bởi kẻ tấn công (Mallory), lý thuyết này hoàn toàn sụp đổ.

Quá trình tấn công Kẻ đứng giữa (Man-in-the-Middle) diễn ra âm thầm và chia cắt hoàn toàn kênh liên lạc của hai nạn nhân mà họ không hề hay biết. Mallory đóng vai trò là một trạm trung chuyển độc hại, thao túng toàn bộ luồng giao tiếp. Diễn biến chi tiết được trình bày trong Bảng 2.1.

Table 2.1: Kịch bản tấn công MITM trên quá trình trao đổi khóa công khai thuần túy

Alice	Mallory (Kẻ tấn công)	Bob
1. Gửi yêu cầu: "Bob, gửi cho tôi $K_{\text{pub_Bob}}$ "	$\xrightarrow{\text{Đánh chặn yêu cầu}}$ Chuyển tiếp yêu cầu đến Bob.	Nhận yêu cầu từ "Alice".
	$\xleftarrow{\text{Đánh chặn } K_{\text{pub_Bob}}}$ Lưu giữ $K_{\text{pub_Bob}}$.	2. Phản hồi: Gửi $K_{\text{pub_Bob}}$ cho Alice.
3. Nhận $K_{\text{pub_Mallory}}$. Định ninh đây là khóa của Bob.	$\xrightarrow{\text{Gửi } K_{\text{pub_Mallory}}}$ Đóng giả Bob, gửi khóa công khai của chính mình cho Alice.	
4. Mã hóa tài liệu mật M : $C = \text{Enc}(K_{\text{pub_Mallory}}, M)$ Gửi C cho Bob.	$\xrightarrow{\text{Đánh chặn bản mã } C}$ Giải mã bằng khóa bí mật của Mallory: $M = \text{Dec}(K_{\text{priv_Mallory}}, C)$. (Mallory đã đọc/sửa được dữ liệu)	
	$\xrightarrow{\text{Mã hóa lại và Gửi}}$ $C' = \text{Enc}(K_{\text{pub_Bob}}, M)$ Gửi C' cho Bob.	5. Nhận C' . Giải mã bằng $K_{\text{priv_Bob}}$. Định ninh dữ liệu đến từ Alice và hoàn toàn bảo mật.

Hậu quả của đòn tấn công này là thảm họa. Mặc dù thuật toán mã hóa (ví dụ RSA) không hề bị bẻ khóa, nhưng tính bảo mật đã bị vô hiệu hóa hoàn toàn do Alice đã sử dụng "nhầm" chìa khóa để khóa cửa. Kịch bản này không chỉ đúng với việc mã hóa dữ liệu mà còn áp dụng tương tự với việc xác minh chữ ký số: Mallory có thể thay thế khóa công khai, từ đó tự do giả mạo chữ ký mang tên Bob mà Alice vẫn xác minh thành công.

Vấn đề Ràng buộc Danh tính (The Identity Binding Problem)

Cội nguồn của đòn tấn công MITM nêu trên xuất phát từ một thiếu sót nền tảng của mật mã khóa công khai: **Sự vắng mặt của tính ràng buộc danh tính (Identity Binding)**.

Về bản chất toán học, một khóa công khai (như K_{pub}) chỉ là một tập hợp các con số vô tri vô giác (ví dụ: một cặp số (e, n) trong RSA, hay một tọa độ điểm (x, y) trên đường

cong Elliptic). Bản thân các con số này không chứa đựng bất kỳ siêu dữ liệu (metadata) nào để chứng minh nó thuộc về thực thể nào. Không có bất kỳ nhãn dán nào trên khóa công khai ghi rằng "*Khóa này là tài sản hợp pháp của Bob, nhân viên ngân hàng X*".

Trong thế giới thực, để chứng minh một bức ảnh thuộc về một cái tên, chúng ta cần Chứng minh nhân dân hoặc Hộ chiếu — một tài liệu do cơ quan nhà nước có thẩm quyền cấp phép, đóng dấu giáp lai để **ràng buộc** (bind) khuôn mặt với đặc điểm nhân thân.

Trên không gian mạng, nếu Alice nhận được một chuỗi bit dài 2048-bit từ một email có tên "Bob", làm sao cô ấy có thể tin tưởng 100% đó không phải là Mallory giả mạo? Nếu không có một cơ chế kiểm chứng độc lập, mọi giao dịch dựa trên mật mã bất đối xứng đều được xây dựng trên một nền móng cát: niềm tin mù quáng vào kênh truyền tải.

Cầu nối tới Hạ tầng Khóa Công khai (PKI) Để mật mã bất đối xứng có thể vận hành trong thế giới thực, nơi mà mọi nút mạng đều có thể là kẻ thù, chúng ta bắt buộc phải có một "Sổ tư pháp" kỹ thuật số. Cần có một cơ chế, một định dạng tiêu chuẩn, và một Bên thứ ba đáng tin cậy (Trusted Third Party) đứng ra bảo lãnh cho tính chính danh của các khóa công khai.

Họ phải tạo ra một "tờ giấy thông hành" kỹ thuật số, trong đó niêm phong chặt chẽ **Khóa công khai (Public Key)** cùng với **Danh tính (Identity)** của chủ sở hữu bằng một Chữ ký số không thể làm giả. Cấu trúc đó chính là **Chứng chỉ số (Digital Certificate)**, và hệ sinh thái quản lý toàn bộ vòng đời của các chứng chỉ này chính là lý do **Hạ tầng Khóa Công khai (PKI - Public Key Infrastructure)** ra đời. Đây sẽ là trọng tâm phân tích của Chương 3.

Chapter 3

Cấu trúc hạ tầng khóa công khai PKI

Hạ tầng khóa công khai (Public Key Infrastructure - PKI) thường bị nhầm tưởng chỉ là một tập hợp các thuật toán mã hóa và phần mềm. Tuy nhiên, trên thực tế, PKI là một hệ thống toàn diện, là sự tổ hợp chặt chẽ giữa con người, quy trình, phần mềm, phần cứng và các chính sách an toàn thông tin. Mục tiêu cốt lõi của hệ thống này là tạo ra, quản lý, phân phối, sử dụng, lưu trữ và thu hồi các chứng chỉ số một cách an toàn, từ đó thiết lập và duy trì nền tảng tin cậy cho các giao dịch điện tử trên môi trường mạng không an toàn.

3.1 Các mô hình phân phối khóa công khai

Sự phát triển của các cơ chế phân phối khóa công khai trải qua nhiều giai đoạn tiến hóa, từ các phương pháp thủ công, đơn giản như Host-based Trust, phân tán như Web of Trust, cho đến các mô hình có tính hệ thống cao như Hierarchical PKI. Mỗi mô hình ra đời nhằm giải quyết những hạn chế của mô hình trước đó, nhưng đồng thời cũng phải đối mặt với những thách thức mới.

3.1.1 Bài toán phân phối khóa công khai

Trong hệ mật mã bất đối xứng, khóa công khai (Public Key) được thiết kế để công bố rộng rãi cho bất kỳ ai muốn giao tiếp an toàn với chủ sở hữu của nó. Tuy nhiên, bài toán đặt ra là làm thế nào để phân phối khóa công khai này rộng rãi mà vẫn đảm bảo tính an toàn. Quá trình phân phối này cần hướng tới việc hỗ trợ bốn mục tiêu an toàn thông tin cơ bản: tính xác thực (Authentication), tính bảo mật (Confidentiality), tính toàn vẹn (Integrity) và tính chống chối bỏ (Non-repudiation).

3.1.2 Public Announcement

Đây là phương pháp cơ bản nhất, trong đó khóa công khai được chia sẻ công khai trên một nền tảng dễ tiếp cận. Một ví dụ tiêu biểu là DNSSEC, nơi khóa công khai được lưu

trữ trực tiếp trong các bản ghi DNS và bất kỳ ai cũng có thể truy vấn thông qua các công cụ như `dig`. Tuy nhiên, điểm yếu lớn nhất của phương pháp này nằm ở khả năng xác thực (Authentication). Khi nhận được một khóa công khai, người dùng không có cơ chế đáng tin cậy nào để kiểm chứng xem khóa đó có thực sự thuộc về thực thể mà nó tự nhận hay không, dẫn đến rủi ro cao về các cuộc tấn công trung gian (Man-in-the-Middle).

3.1.3 Host-based Trust

Mô hình này dựa trên việc thiết lập sự tin cậy thủ công giữa các thiết bị. Điển hình là cơ chế của SSH, sử dụng tệp `known_hosts` để mỗi máy tính tự lưu trữ và quản lý danh sách các khóa công khai mà nó cho là an toàn. Mặc dù hiệu quả trong các hệ thống nhỏ, nhưng khi số lượng thiết bị tăng lên, số lượng kết nối tin cậy cần thiết lập sẽ tăng theo cấp số nhân ($O(n^2)$ cho n máy tính). Điều này khiến mô hình Host-based Trust hoàn toàn thiếu khả năng mở rộng (scalability) và không thể áp dụng cho mạng Internet toàn cầu.

3.1.4 Web of Trust (PGP)

Để khắc phục tính tập trung, Web of Trust (WoT) được giới thiệu trong các hệ thống như PGP (Pretty Good Privacy) [3]. Đây là một mô hình phi tập trung, dựa trên sự xác nhận lẫn nhau (peer endorsement) giữa các người dùng; mỗi cá nhân tự chịu trách nhiệm quyết định mức độ tin cậy đối với khóa của người khác dựa trên các chữ ký xác nhận của bạn bè hoặc đối tác. Dù rất dân chủ và phù hợp cho các cộng đồng kỹ thuật nhỏ, WoT gặp phải vấn đề lớn về "bootstrapping" (khó khăn cho người mới tham gia để có được sự tin cậy) và tính thân thiện với người dùng thông thường, do đó không thể mở rộng ở quy mô toàn cầu.

3.1.5 Hierarchical PKI

Đây là mô hình phổ biến và thành công nhất hiện nay trên Internet. Hierarchical PKI sử dụng cấu trúc cây phân cấp dựa trên nguyên tắc "tin cậy bắc cầu" (transitive trust). Sự tin cậy được kế thừa từ các Root CA (Tổ chức phát hành chứng chỉ gốc) trên cùng, truyền xuống các Intermediate CA, và cuối cùng đến các chứng chỉ thực thể (Leaf certificates). Ở phía máy khách, hệ điều hành và trình duyệt (như Mozilla NSS, Microsoft, Apple) duy trì một kho lưu trữ chứng chỉ gốc đáng tin cậy (Trust Store). Hoạt động của các CA này được quản lý và tiêu chuẩn hóa thông qua các yêu cầu cơ sở (baseline requirements) do CA/Browser Forum [4] ban hành.

3.1.6 So sánh trade-off

Sự khác biệt giữa các mô hình chủ yếu nằm ở cách tiếp cận quản lý sự tin cậy. Mô hình phân cấp (Hierarchical) có ưu điểm là dễ dàng kiểm toán (audit) và thu hồi chứng chỉ

(revoke), quản lý thống nhất, nhưng lại biến các Root CA thành những điểm yếu chí mạng (Single Point of Failure - SPOF) về mặt tin cậy. Ngược lại, mô hình phi tập trung như Web of Trust không có SPOF, nhưng quá trình đánh giá niềm tin lại không đủ tính bắc cầu mạnh mẽ và quá phức tạp đối với người dùng phổ thông.

Table 3.1: So sánh các mô hình tin cậy (Trust Models)

Tiêu chí	Hierarchical PKI (Centralized)	Web of Trust (Decentralized)
Cấu trúc	Cây phân cấp từ Root CA xuống Leaf	Mạng lưới đồng đẳng (Peer-to-peer)
Quản lý thu hồi	Dễ dàng, quản lý tập trung (CRL/OCSP)	Khó khăn, phụ thuộc vào bản cập nhật của từng node
Rủi ro (SPOF)	Root CA bị xâm nhập là thảm họa	Không có SPOF rõ rệt
Tính thân thiện	Rất cao (trong suốt với người dùng)	Thấp (đòi hỏi kiến thức kỹ thuật)

Hình 3.1 tóm tắt mô hình phân cấp phổ biến nhất trên Web PKI: niềm tin được kế thừa từ Root CA xuống Intermediate CA và chứng chỉ leaf.

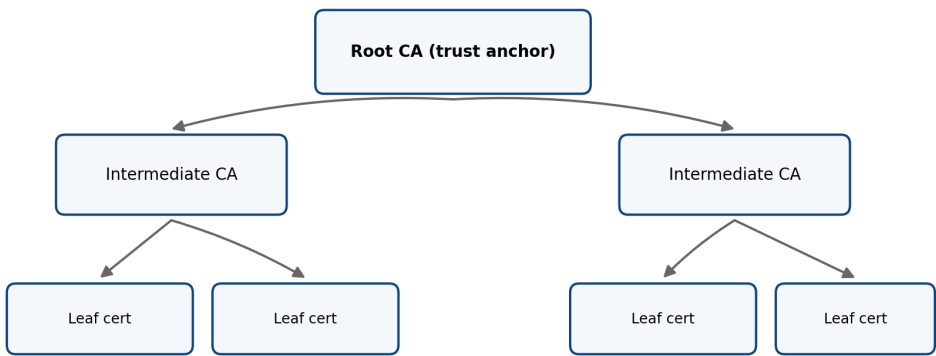


Figure 3.1: Mô hình phân cấp Hierarchical PKI và chuỗi tin cậy

3.2 Định nghĩa và mục tiêu PKI

Như đã đề cập, Hạ tầng khóa công khai (PKI) là một bộ khung bao gồm các thành phần công nghệ (phần cứng, phần mềm), cơ cấu tổ chức (con người), và các tài liệu điều chỉnh (chính sách, quy trình) nhằm mục đích phát hành, duy trì, và thu hồi các chứng chỉ số số trong một môi trường mạng mở.

Mục tiêu lớn nhất của PKI là giải quyết bài toán cốt lõi về "sự tin cậy" (trust) trên môi trường số bằng cách cung cấp một cơ chế bảo đảm để gắn kết (bind) một danh tính

thực (một con người, một tổ chức, hoặc một thiết bị) với một khóa công khai cụ thể thông qua bên thứ ba được tin tưởng là CA. Quan trọng hơn, PKI không chỉ dừng lại ở các giao thức kỹ thuật, mà nó mang đậm tính chất quản trị (governance) – giải quyết các vấn đề vĩ mô như: ai có đủ thẩm quyền để làm CA? Tiêu chuẩn nào CA phải tuân thủ để được đưa vào Trust Store của các trình duyệt?

****Ứng dụng của public key: Chữ ký số**** Sau khi PKI đã thiết lập sự tin cậy trong khóa công khai của một thực thể (thông qua việc cấp chứng chỉ số), thực thể đó có thể sử dụng cặp khóa của mình để đạt được các mục tiêu an toàn thông tin cụ thể, như được minh họa trong Hình 3.2. Sơ đồ này cho thấy quy trình tạo và xác thực chữ ký số, giúp đảm bảo tính toàn vẹn (Integrity) và tính xác thực (Authentication) của thông điệp được truyền đi.

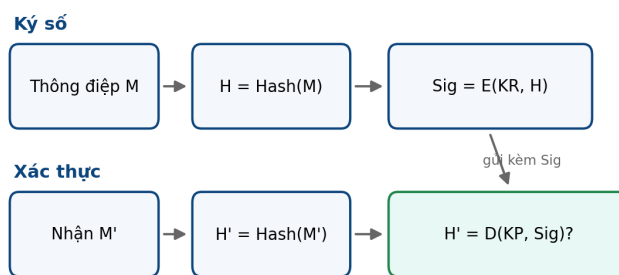


Figure 3.2: Quy trình tạo và xác thực chữ ký số (hash + khóa bất đối xứng)

Sử dụng khóa bí mật (Private Key) để ký số lên hàm băm (Message Digest) tạo ra một bằng chứng không thể phủ nhận về nguồn gốc của thông điệp. Khi người nhận dùng khóa công khai tương ứng để xác thực thành công, họ biết chắc rằng thông điệp thực sự đến từ chủ sở hữu khóa và không bị chỉnh sửa trên đường truyền. Đây chính là một trong những mục tiêu hoạt động then chốt mà PKI hướng tới nhằm tạo ra sự tin cậy tổng thể.

3.3 Các thành phần PKI

CA – Certificate Authority

Certificate Authority (Tổ chức phát hành chứng chỉ) là trái tim của hệ thống PKI. CA chịu trách nhiệm chính trong việc phát hành, thu hồi chứng chỉ và duy trì danh sách chứng chỉ bị thu hồi (CRL). Để tối ưu hóa tính bảo mật, các CA thường được tổ chức theo cấu trúc phân tầng. Trên cùng là Root CA, nắm giữ khóa quan trọng nhất và thường được giữ ở trạng thái ngoại tuyến (offline, air-gapped) để tránh bị tấn công mạng. Root CA ký cấp phát chứng chỉ cho các Intermediate CA (CA trung gian), và các Intermediate CA này mới là đơn vị trực tiếp tương tác và cấp phát chứng chỉ cho thực thể cuối (Leaf/End-entity).

Quy trình cơ bản bao gồm: người dùng tạo Yêu cầu ký chứng chỉ (CSR) → CA xác minh danh tính → CA dùng khóa bí mật để ký chứng chỉ → Công bố chứng chỉ.

RA – Registration Authority

Registration Authority (Tổ chức đăng ký) hoạt động như một cơ quan tiền phương cho CA. Việc tách bạch quá trình xác minh danh tính (thực hiện bởi RA) khỏi quá trình ký chứng chỉ (thực hiện bởi CA) giúp phân quyền trách nhiệm và giảm thiểu rủi ro bảo mật cho hệ thống ký. RA đóng vai trò kiểm tra tính hợp lệ của người nộp đơn. Ví dụ trong thực tế, RA có thể yêu cầu người dùng xuất trình Căn cước công dân hoặc Hộ chiếu để đối chiếu; sau khi RA xác nhận hợp lệ, yêu cầu mới được chuyển tiếp để CA thực hiện thao tác ký số sinh ra chứng chỉ.

Repository – Kho lưu trữ chứng chỉ

Kho lưu trữ (Repository) là cơ sở dữ liệu công khai lưu trữ các chứng chỉ đang có hiệu lực và danh sách các chứng chỉ đã bị thu hồi. Kho này thường được triển khai trên các máy chủ thư mục LDAP (Lightweight Directory Access Protocol) hoặc máy chủ HTTP/HTTPS, đảm bảo các client có thể truy vấn nhanh chóng để lấy chứng chỉ của người cần giao tiếp hoặc kiểm tra trạng thái CRL bất cứ lúc nào.

CRL – Certificate Revocation List

Danh sách chứng chỉ bị thu hồi (CRL) là một tài liệu được CA ký số và phát hành định kỳ, liệt kê số seri của các chứng chỉ đã bị mất hiệu lực trước khi hết hạn. Các lý do thu hồi (reason codes) thường được đi kèm, ví dụ như lộ khóa bí mật (**keyCompromise**) hoặc bản thân CA bị xâm phạm (**CACompromise**). Hạn chế lớn của CRL là kích thước tệp có thể phình to rất nhanh và thông tin không được cập nhật tức thời (có thể lỗi thời - stale trong nhiều giờ hoặc vài ngày trước khi bản mới được tải về). Để giảm tải mạng, giải pháp **delta CRL** [5] được sử dụng để chỉ tải xuống những thay đổi so với bản CRL đầy đủ trước đó.

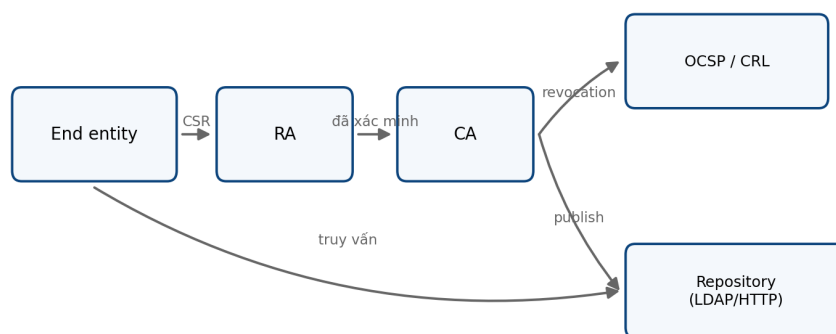
OCSP – Online Certificate Status Protocol

Giao thức kiểm tra trạng thái chứng chỉ trực tuyến (OCSP) [6] ra đời nhằm khắc phục độ trễ của CRL. Thay vì tải toàn bộ danh sách, client gửi truy vấn chứa số seri của chứng chỉ đến máy chủ OCSP Responder và nhận về phản hồi theo thời gian thực (Realtime) về trạng thái hợp lệ của chứng chỉ đó. Để giải quyết vấn đề về quyền riêng tư (vì CA biết client đang truy cập trang web nào) và độ trễ (latency) khi client phải tự liên hệ CA, kỹ thuật **OCSP Stapling** [7] được áp dụng. Máy chủ web sẽ tự động lấy (pre-fetch) phản hồi OCSP có chữ ký của CA theo định kỳ, sau đó "gắn kèm" (staple) bằng chứng này vào quá trình thiết lập kết nối TLS ban đầu với client.

Table 3.2: So sánh các phương pháp kiểm tra trạng thái thu hồi chứng chỉ

Tiêu chí	CRL	OCSP	OCSP Stapling
Độ trễ cập nhật	Cao (phụ thuộc chu kỳ tải)	Thấp (thời gian thực)	Thấp (thời gian thực, có cache)
Chi phí băng thông	Rất cao (tải toàn bộ list)	Thấp	Rất thấp
Quyền riêng tư	Tốt (Client kiểm tra cục bộ)	Kém (CA biết lịch sử duyệt web)	Tốt (Client không gọi trực tiếp CA)

Hình 3.3 mô tả luồng tương tác giữa các thành phần chính trong một triển khai PKI điển hình.

**Figure 3.3:** Các thành phần PKI và luồng cấp phát, lưu trữ, kiểm tra chứng chỉ

3.4 Chu kỳ sống của chứng chỉ

Hoạt động của PKI phụ thuộc vào việc quản lý chặt chẽ vòng đời (lifecycle) của một chứng chỉ số. Chu kỳ sống này bao gồm các giai đoạn tuần tự:

- 1. Tạo cặp khóa (Keypair Generation):** Thực thể tự tạo Khóa công khai và Khóa bí mật một cách an toàn.
- 2. Tạo yêu cầu (CSR Creation):** Gói Khóa công khai và các thông tin định danh vào một bản tin CSR (Certificate Signing Request).
- 3. Xác minh (Verification):** RA/CA kiểm tra tính hợp lệ của thông tin và quyền sở hữu tên miền/tổ chức.
- 4. Phát hành (Issuance):** CA ký điện tử lên CSR để tạo ra chứng chỉ số hợp lệ.
- 5. Sử dụng (Usage):** Chứng chỉ được triển khai trên hệ thống (web server, email) để thực hiện mã hóa và xác thực.

6. **Gia hạn / Thu hồi (Renewal / Revocation):** Trước khi hết hạn, chứng chỉ cần được gia hạn. Nếu khóa bí mật bị lộ hoặc thông tin không còn đúng, chứng chỉ sẽ bị đưa vào quá trình thu hồi (đưa vào CRL hoặc báo trạng thái qua OCSP).

Với sự bùng nổ của quy mô dịch vụ đám mây và microservices, việc cấu hình thủ công không còn khả thi. Tự động hóa vòng đời chứng chỉ (Certificate Lifecycle Automation) thông qua các giao thức như ACME (được Let's Encrypt sử dụng phổ biến), SCEP, hoặc EST ngày càng đóng vai trò quan trọng yếu giúp đảm bảo hệ thống không bị gián đoạn dịch vụ do chứng chỉ hết hạn.



Figure 3.4: Chu kỳ sống của chứng chỉ số trong hệ thống PKI

Chapter 4

Chứng chỉ số và các chuẩn

Chứng chỉ số là cầu nối liên kết một định danh với khóa công khai tương ứng, cho phép các thực thể độc lập thiết lập kênh giao tiếp an toàn mà không cần chia sẻ bí mật từ trước. Mọi hạ tầng PKI đều xoay quanh vòng đời của đối tượng này: từ lúc khởi tạo, phân phối cho đến khi thu hồi.

Trong hệ sinh thái hiện đại, X.509 đã trở thành khuôn dạng chuẩn mực thực tế. Một chứng chỉ X.509 không chỉ lưu trữ khóa công khai mà còn mang theo chữ ký của Certificate Authority (CA) cùng hệ thống chính sách vận hành. Chương này phân tích cấu trúc của X.509, quy trình cấp phát qua Certificate Signing Request (CSR) và các chuẩn định dạng liên quan như họ PKCS.

4.1 Chứng chỉ số X.509

X.509 ban đầu là một phần của hệ thư mục X.500 do ITU-T thiết kế. Khi Internet phát triển, IETF đã thu hẹp và chuẩn hóa khuôn dạng này thành hồ sơ PKIX thông qua RFC 5280 [5]. Hồ sơ này định nghĩa chính xác cách các ứng dụng Web PKI xử lý chứng chỉ và danh sách thu hồi (CRL).

Khuôn dạng X.509 đã trải qua ba phiên bản:

- **Phiên bản 1:** Chứa các trường cơ sở như số seri, thuật toán, thời hạn và tên.
- **Phiên bản 2:** Thêm định danh duy nhất cho issuer và subject để xử lý trùng lặp tên.
- **Phiên bản 3:** Đưa vào cơ chế extension (phần mở rộng), biến X.509 thành một bộ khung linh hoạt. Nhờ extension, cùng một định dạng có thể phục vụ chứng chỉ root CA, máy chủ TLS, thư điện tử hay chữ ký mã.

Một chứng chỉ X.509 gồm ba phần ở mức ASN.1. Phần `tbsCertificate` là dữ liệu “to be signed”: CA ký đúng phần này. Phần thuật toán chữ ký cho biết thuật toán được dùng, chẳng hạn RSA với SHA-256 hoặc ECDSA với SHA-256. Phần giá trị chữ ký chứa

chữ ký trên `tbsCertificate`. Cấu trúc này cho phép bên kiểm tra tái tạo dữ liệu đã ký và xác minh bằng khóa công khai của issuer.

Hình 4.1 tóm tắt ba phần ASN.1 cốt lõi và các trường quan trọng trong `tbsCertificate`.

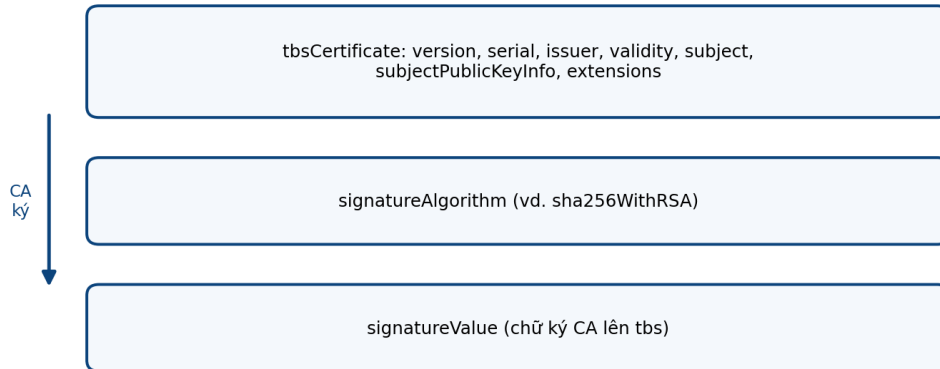


Figure 4.1: Cấu trúc logic của chứng chỉ X.509 v3

X.509 được mã hóa theo DER trong tầng nhị phân. DER là một dạng mã hóa xác định của ASN.1, nên cùng một cấu trúc dữ liệu có đúng một biểu diễn byte hợp lệ. PEM là lớp bao gói văn bản: dữ liệu DER được mã hóa Base64 và đặt giữa các dòng `--BEGIN CERTIFICATE--` và `---END CERTIFICATE---`. Vì vậy, DER và PEM không phải hai loại chứng chỉ khác nhau. Chúng là hai cách biểu diễn cùng một nội dung.

Table 4.1: Các trường chính trong chứng chỉ X.509 v3

Trường	Ý nghĩa	Ghi chú
<code>version</code>	Phiên bản khuôn dạng chứng chỉ.	Web PKI thực tế dùng v3 để hỗ trợ extension.
<code>serialNumber</code>	Số seri do CA gán trong phạm vi issuer.	Được dùng khi thu hồi qua CRL hoặc OCSP.
<code>issuer</code>	DN của thực thể ký chứng chỉ.	Thường là CA trung gian hoặc root CA.
<code>validity</code>	Cặp thời điểm <code>notBefore</code> và <code>notAfter</code> .	Chứng chỉ chỉ hợp lệ trong khoảng này.
<code>subject</code>	DN của chủ thể được cấp chứng chỉ.	Có thể rỗng nếu danh tính nằm trong SAN.
<code>subjectPublicKeyInfo</code>	Thuật toán khóa công khai và giá trị khóa.	Ví dụ RSA, ECDSA hoặc EdDSA.
<code>extensions</code>	Các ràng buộc và thuộc tính mở rộng.	Là cơ chế trung tâm của X.509 v3.

4.2 Các trường X.509 v3

Dù có nhiều phiên bản, X.509 v3 xoay quanh một số trường dữ liệu cốt lõi chi phối vòng đời và độ an toàn của chứng chỉ:

- **Định danh và Phiên bản:** `version` quyết định cách giải mã dữ liệu, trong khi `serialNumber` giúp CA quản lý và thu hồi chứng chỉ.
- **Thực thể liên quan:** `issuer` lưu tên CA phát hành, còn `subject` định danh thực thể sở hữu chứng chỉ.
- **Hiệu lực (validity):** Giới hạn bằng hai mốc `notBefore` và `notAfter`. Bên kiểm tra sẽ từ chối giao dịch nếu thời gian thực tế nằm ngoài khoảng này.
- **Khóa công khai (subjectPublicKeyInfo):** Chứa giá trị khóa và định danh thuật toán (như RSA, ECDSA). Tùy vào giao thức phía trên, khóa này được dùng để xác thực chữ ký hoặc thiết lập kênh trao đổi bảo mật.
- **Phần mở rộng (extensions):** Nơi chứa phần lớn các quy tắc kiểm tra an toàn trong thực tiễn.

4.3 Distinguished Name

Distinguished Name (DN) là định danh phân cấp thừa hưởng từ mô hình X.500. Trong chứng chỉ, DN được dùng để định danh `issuer` và `subject`. Một DN cấu thành từ các thuộc tính cơ bản:

C (Country): Mã quốc gia chuẩn ISO.

ST (State/Province): Tỉnh hoặc bang.

L (Locality): Thành phố hoặc địa phương.

O (Organization): Tên tổ chức.

OU (Organizational Unit): Đơn vị trực thuộc tổ chức.

CN (Common Name): Tên gọi phổ thông.

Cấu trúc này hữu ích trong môi trường doanh nghiệp vì nó phân định rõ `subject` theo cây phân cấp.

Trong HTTPS, **CN** không còn là nơi chính để kiểm tra tên miền. RFC 6125 yêu cầu ứng dụng ưu tiên `subject alternative name` và không quay lại **CN** khi SAN thuộc loại tên phù hợp đã có mặt [8]. Điều này tách rõ tên hiển thị trong DN khỏi tập tên mà chứng chỉ thực sự chứng nhận. Với chứng chỉ máy chủ, SAN mới là nơi chứa DNS name hoặc IP address được so khớp với tên mà client truy cập.

4.4 Extension quan trọng

Extension làm cho X.509 v3 phù hợp với nhiều ngữ cảnh sử dụng. Mỗi extension có định danh OID, giá trị và cờ critical. Nếu một extension critical không được hiểu, bên kiểm tra phải từ chối chứng chỉ theo RFC 5280 [5]. Cơ chế này cho phép CA áp đặt điều kiện an toàn mà client không được bỏ qua.

Table 4.2: Một số extension thường gặp trong X.509 v3

Extension	Nội dung	Vai trò
BasicConstraints	Cờ cA và tùy chọn pathLenConstraint.	Phân biệt CA với end-entity certificate.
KeyUsage	Bit như digitalSignature, keyEncipherment, keyCertSign.	Giới hạn thao tác mật mã được phép.
ExtendedKeyUsage	OID mục đích như server authentication, client authentication hoặc code signing.	Thu hẹp ngữ cảnh sử dụng của khóa.
SubjectAltName	DNS name, IP address, email hoặc URI của subject.	Cơ sở kiểm tra tên miền trong Web PKI.
SubjectKeyIdentifier	Định danh suy ra từ khóa công khai của subject.	Hỗ trợ tìm và nối chuỗi chứng chỉ.
AuthorityKeyIdentifier	Định danh khóa của issuer.	Ghép chứng chỉ với CA đã ký nó.
CRLDistributionPoints	Vị trí tải CRL.	Cho phép client kiểm tra thu hồi bằng CRL.
AuthorityInfoAccess	Vị trí OCSP responder hoặc chứng chỉ issuer.	Hỗ trợ kiểm tra trạng thái và dựng chuỗi.

Các extension này chia thành ba nhóm chức năng chính:

- **Phân quyền và Mục đích:** BasicConstraints đóng vai trò then chốt để phân biệt chứng chỉ CA và chứng chỉ end-entity. Cùng với KeyUsage (giới hạn thao tác mật mã cấp thấp như ký số) và ExtendedKeyUsage (giới hạn ngữ cảnh ứng dụng như server authentication), chúng ngăn chặn việc lạm dụng khóa sai mục đích.
- **Định danh chủ thể:** Khác với thông tin chung trong DN, SubjectAltName (SAN) trực tiếp mang các DNS name hoặc IP address cần bảo vệ. Đây là cơ sở chính để trình duyệt đối chiếu với địa chỉ website.
- **Hỗ trợ xác thực chuỗi:** SubjectKeyIdentifier và AuthorityKeyIdentifier giúp client chọn đúng CA khi có nhiều chứng chỉ trùng tên. Đồng thời, CRLDistributionPoints và AuthorityInfoAccess chỉ dẫn phần mềm nơi lấy thông tin thu hồi qua CRL hoặc OCSP [6].

4.5 Certificate Signing Request

Certificate Signing Request (CSR) là yêu cầu ký chứng chỉ do chủ thể gửi cho CA. Khuôn dạng CSR phổ biến là PKCS #10, được RFC 2986 mô tả [9]. CSR chứa thông tin subject, khóa công khai, một tập thuộc tính tùy chọn và chữ ký do chính khóa bí mật ứng với khóa công khai trong CSR tạo ra. Chữ ký này chứng minh rằng người gửi CSR có quyền kiểm soát khóa bí mật, nhưng nó không chứng minh danh tính pháp lý hoặc quyền sở hữu tên miền. Phần đó thuộc trách nhiệm xác minh của RA hoặc CA.

Quy trình cấp phát chứng chỉ diễn ra theo bốn bước:

1. Subject tự sinh cặp khóa và bảo vệ khóa bí mật cục bộ.
2. Subject tạo CSR bằng khóa công khai và thông tin định danh mong muốn, sau đó ký CSR bằng khóa bí mật.
3. CA tiếp nhận CSR và kiểm tra quyền kiểm soát tên miền hoặc tiêu chí của loại chứng chỉ tương ứng.
4. CA phát hành chứng chỉ X.509 bằng cách ký lên dữ liệu đã xác minh và gửi lại cho subject.

Trong toàn bộ luồng vận hành này, khóa bí mật không bao giờ rời khỏi hệ thống của subject.

4.6 Phân loại chứng chỉ

CA/Browser Forum Baseline Requirements quy định mức tiêu chuẩn tối thiểu để một chứng chỉ máy chủ TLS được các trình duyệt tin cậy [4]. Tùy vào quy trình xác minh danh tính, chứng chỉ TLS chia làm ba nhóm chính:

Domain Validation (DV): CA chỉ kiểm tra subject có quyền kiểm soát tên miền hay không. Quá trình này hoàn toàn tự động.

Organization Validation (OV): CA kiểm tra thêm sự tồn tại pháp lý của tổ chức.

Extended Validation (EV): CA thực hiện các bước định danh tổ chức nghiêm ngặt dựa trên quy tắc EV Guidelines [10].

Wildcard certificate chỉ bao phủ một cấp nhân theo cách diễn giải thông thường của Web PKI. Ví dụ, `*.example.com` có thể khớp `www.example.com`, nhưng không khớp `a.b.example.com`. Multi-domain certificate dùng SAN để chứa nhiều tên cụ thể. Cách này phù hợp khi một dịch vụ có nhiều tên miền hoặc khi cần hợp nhất chứng chỉ cho các endpoint liên quan.

Table 4.3: Phân loại chứng chỉ theo mục đích sử dụng

Loại	Đặc điểm	Cách dùng
DV	Xác minh quyền kiểm soát tên miền.	Website công khai, tự động hóa cấp phát.
OV	Xác minh thêm thông tin tổ chức.	Dịch vụ cần hiện thông tin pháp nhân rõ hơn.
EV	Xác minh tổ chức theo yêu cầu EV.	Môi trường cần quy trình nhận diện chặt hơn.
Wildcard	Tên dạng *.example.com.	Nhiều subdomain cùng một cấp.
Multi-domain/SAN	Nhiều tên trong SubjectAltName.	Gom nhiều DNS name vào một chứng chỉ.
Code signing	EKU cho ký mã hoặc chính sách tương ứng.	Ký phần mềm, script hoặc gói cài đặt.
S/MIME	Gắn khóa công khai với địa chỉ email.	Ký và mã hóa thư điện tử [11].
Client certificate	Subject đại diện người dùng, thiết bị hoặc dịch vụ.	Xác thực client trong VPN, mTLS hoặc hệ nội bộ.

4.7 Các chuẩn PKCS

Trong khi X.509 định nghĩa bản thân chứng chỉ, thực tế triển khai cần thêm các chuẩn định dạng để đóng gói, vận chuyển và sử dụng chứng chỉ một cách an toàn. Public-Key Cryptography Standards (PKCS) do RSA Laboratories khởi xướng đóng vai trò lấp đầy khoảng trống này.

Dù họ PKCS có rất nhiều thành phần, hạ tầng PKI hiện đại chủ yếu dựa vào ba chuẩn cốt lõi:

- **PKCS #7 (CMS):** Quy định cú pháp bao gói dữ liệu mật mã. IETF đã tiếp quản và phát triển chuẩn này thành Cryptographic Message Syntax (CMS) qua RFC 5652 [12]. CMS chính là nền tảng cho S/MIME và nhiều giao thức ký số.
- **PKCS #10:** Định dạng tiêu chuẩn cho Certificate Signing Request, quy định cách subject đóng gói thông tin và khóa công khai để gửi cho CA [9] (xem Mục 4.5).
- **PKCS #12:** Định dạng tệp kho lưu trữ mật mã [13]. Một tệp .p12 hoặc .pfx có thể chứa đồng thời khóa bí mật, chứng chỉ cá nhân và toàn bộ chuỗi chứng chỉ CA, thường được mã hóa bảo vệ bằng mật khẩu để di chuyển giữa các hệ thống.

4.8 Các chuẩn hóa và quy tắc vận hành

Web PKI không hoạt động dựa trên một tài liệu đơn lẻ mà là kết quả hợp nhất từ các chuẩn kỹ thuật IETF và quy tắc quản trị thực tiễn:

- **Cấu trúc cốt lõi:** RFC 5280 là xương sống của hồ sơ Internet X.509, quy định từ định dạng chứng chỉ đến thuật toán xác thực đường dẫn [5]. Nhóm chuẩn PKCS bao bọc các thao tác yêu cầu cấp phát (RFC 2986) và vận chuyển an toàn (RFC 5652, RFC 7292).
- **Kiểm tra trạng thái minh bạch:** RFC 6960 chuẩn hóa giao thức OCSP để truy vấn trạng thái thu hồi theo thời gian thực [6]. Bên cạnh đó, RFC 6962 và RFC 9162 định hình cơ chế Certificate Transparency, buộc chứng chỉ TLS phải được ghi log công khai vào cây Merkle để chống phát hành sai trái [14], [15].
- **Quy tắc vận hành:** CA/Browser Forum Baseline Requirements áp đặt tiêu chuẩn nghiệp vụ và an toàn hệ thống cho các CA [4]. Một CA có thể tuân thủ hoàn hảo RFC về mặt kỹ thuật, nhưng nếu vi phạm bộ quy tắc này, root certificate của họ sẽ bị các hãng trình duyệt gỡ bỏ.

Chapter 5

Triển khai và ứng dụng thực tế

Bỏ qua các chi tiết cấu trúc tĩnh của X.509, chương này tập trung vào khía cạnh vận hành động: cách PKI nhúng vào các giao thức bảo mật thực tế. Dù là duyệt web qua HTTPS, gửi email mã hóa, hay ký kết văn bản điện tử, hạt nhân của niềm tin luôn nằm ở việc kiểm tra chuỗi chứng chỉ, trạng thái thu hồi và ràng buộc mục đích sử dụng.

Table 5.1: So sánh một số ứng dụng PKI

Ứng dụng	Đối tượng xác thực	Loại chứng chỉ	Kiểm tra chính	Rủi ro vận hành
HTTPS/TLS	Máy chủ dịch vụ	TLS server certificate	Chuỗi CA, SAN, thời hạn, KU/EKU, thu hồi nếu có	Sai hostname, thiếu intermediate CA, khóa máy chủ bị lộ.
Ký số tài liệu	Người ký hoặc tổ chức	Chứng chỉ ký số cá nhân, tổ chức hoặc dịch vụ ký từ xa	Chữ ký tài liệu, chuỗi CA, thời điểm ký, trạng thái thu hồi	Không có timestamp tin cậy, khóa ký bị dùng ngoài kiểm soát.
S/MIME	Người gửi và người nhận email	Chứng chỉ gắn với địa chỉ email	Chữ ký email, chứng chỉ người gửi, chứng chỉ người nhận khi mã hóa	Khó quản lý chứng chỉ người dùng, mất khóa giải mã.
VPN/IPSec	Gateway, client hoặc site ngang hàng	Chứng chỉ thiết bị hoặc người dùng	CERT, CERTREQ, AUTH trong IKEv2, chuỗi CA nội bộ	CA nội bộ yếu, cấp chứng chỉ sai danh tính, thu hồi chậm.
Code signing	Nhà phát hành phần mềm	Code signing certificate	Chữ ký mã, certificate chain, timestamp, trạng thái thu hồi	Private key bị lộ, pipeline build bị chiếm quyền.
eGovernment	Công dân, tổ chức, cơ quan, dịch vụ ký số	Chứng chỉ công cộng hoặc chứng chỉ ký số từ xa	Chữ ký văn bản, chứng chỉ người ký, timestamp, OCSF/CRL	Liên thông hệ thống, vòng đời chứng chỉ, trải nghiệm người dùng.

5.1 HTTPS / TLS

Trong giao tiếp web, mã hóa kênh truyền là chưa đủ nếu client không thể xác thực danh tính máy chủ để chống lại tấn công trung gian (MitM). Chứng chỉ TLS giải quyết điều này bằng cách neo khóa công khai của máy chủ vào DNS name thực tế.

Khi bắt tay TLS 1.3, máy chủ gửi chứng chỉ leaf kèm theo các chứng chỉ intermediate (thông điệp `Certificate`) và dùng khóa bí mật để ký thông điệp `CertificateVerify` [16]. Trình duyệt nhận dữ liệu sẽ lần ngược chữ ký từ leaf qua các intermediate CA cho tới

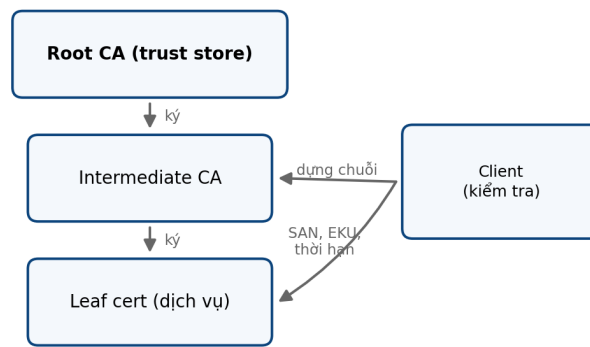


Figure 5.1: Mẫu kiểm tra PKI trong ứng dụng thực tế

khi chạm đến một root CA có sẵn trong trust store của hệ điều hành. Xuyên suốt quá trình đó, client đối chiếu tên miền truy cập với trường SAN, kiểm tra ràng buộc mục đích sử dụng (EKU), và truy vấn trạng thái thu hồi qua OCSP hoặc CRL [6]. Nhờ tích hợp Certificate Transparency [15], mọi chứng chỉ cấp ra đều được ghi nhận vào log công khai, giúp phát hiện sớm các phát hành trái phép.

Điểm yếu thực tiễn thường không nằm ở thuật toán mật mã mà ở cấu hình. Việc máy chủ quên gửi kèm intermediate CA sẽ khiến client không thể dựng chuỗi niềm tin. Ngoài ra, việc quên gia hạn chứng chỉ thường xuyên làm gián đoạn dịch vụ, một vấn đề đã được cải thiện đáng kể nhờ các giao thức tự động hóa cấp phát như ACME [17].

Hình 5.2 mô tả các bước chính trong bắt tay TLS khi máy chủ gửi chứng chỉ và client xác minh chuỗi tin cậy.

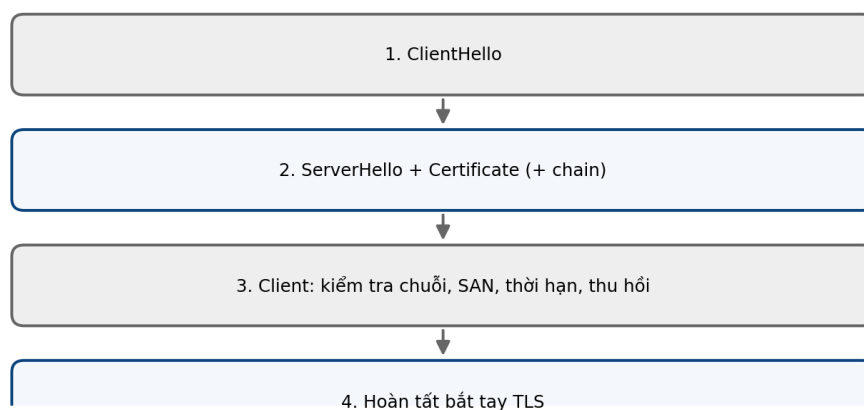


Figure 5.2: Tóm tắt vai trò chứng chỉ TLS trong bắt tay HTTPS

5.2 Chữ ký số văn bản

Việc chèn ảnh chữ ký tay vào tài liệu không mang lại giá trị pháp lý hay mật mã vì nó không chống được sửa đổi nội dung. Chữ ký số giải quyết trọn vẹn yêu cầu này bằng cách liên kết chặt chẽ ba yếu tố: hàm băm của văn bản (đảm bảo toàn vẹn), khóa bí mật của người ký (chống chối bỏ), và chứng chỉ số (định danh người ký).

Thay vì giao tiếp theo thời gian thực như TLS, chữ ký văn bản mang tính lưu trữ lâu dài. Người nhận kiểm tra chữ ký trên tài liệu trước, sau đó mới xác minh tính hợp lệ của chứng chỉ người ký. Tại đây, Timestamp (dấu thời gian) đóng vai trò sinh tử [18]. Nếu tài liệu được đóng dấu bởi một Time Stamping Authority tin cậy, bên kiểm tra có thể khẳng định chữ ký được tạo ra trước khi chứng chỉ hết hạn hoặc bị thu hồi. Điều này giúp bảo vệ giá trị pháp lý của tài liệu theo thời gian, độc lập với vòng đời của chứng chỉ gốc.

Khâu yếu nhất trong ký số văn bản là quản lý khóa bí mật ở phía người dùng. Nếu khóa lưu trên phần mềm mà không có thiết bị bảo vệ cứng (như USB Token hay Smartcard) hoặc hệ thống quản lý an toàn, kẻ tấn công có thể giả mạo chữ ký. Tương tự, nếu tài liệu chỉ dùng thời gian hệ thống cục bộ thay vì timestamp chuẩn hóa, tính xác thực của nó trong tương lai sẽ bị nghi ngờ.

5.3 S/MIME

Hệ thống email thông thường phơi bày nội dung thư trên đường truyền và dễ bị giả mạo địa chỉ. S/MIME giải quyết giới hạn này bằng cách sử dụng chứng chỉ X.509 gắn với địa chỉ email để cung cấp đồng thời cơ chế ký số (xác thực nguồn gốc) và mã hóa (bảo mật nội dung) [11].

Khác với PGP vận hành theo mô hình mạng lưới tin cậy (Web of Trust) phi tập trung, S/MIME dựa hoàn toàn vào hệ thống CA. Khi mã hóa thư, phần mềm gửi cần lấy trước chứng chỉ của tất cả người nhận, sử dụng khóa công khai của họ để bọc khóa phiên bảo vệ nội dung. Ở chiều ngược lại, ứng dụng nhận phải dùng khóa bí mật cục bộ để mở khóa phiên, tháo gỡ từng lớp định dạng MIME đã đóng gói.

Rào cản lớn nhất của S/MIME là vòng đời chứng chỉ cá nhân. Việc di chuyển hay phục hồi khóa bí mật khi người dùng đổi thiết bị rất phức tạp. Một khi khóa bí mật bị mất, người dùng vĩnh viễn mất quyền truy cập vào tất cả các email mã hóa cũ của chính mình. Sự dịch chuyển địa chỉ email hoặc thay đổi định danh trong tổ chức cũng đòi hỏi chính sách ánh xạ cấu hình đặc biệt khắt khe.

5.4 VPN và IPSec

Các kết nối VPN (đặc biệt là IPSec site-to-site hoặc remote access) cần xác thực hai đầu thiết bị trước khi mã hóa luồng dữ liệu. Nếu chỉ dùng khóa chia sẻ trước (Pre-Shared

Key), việc quản lý sẽ nhanh chóng vượt tầm kiểm soát khi mạng lưới mở rộng, đồng thời rất khó thu hồi quyền truy cập của một thiết bị đơn lẻ mà không phải thay đổi cấu hình diện rộng.

Khi tích hợp PKI, giao thức IKEv2 tận dụng chứng chỉ để thực hiện xác thực hai chiều [19]. Mỗi gateway hoặc client giữ một chứng chỉ do CA nội bộ phát hành. Thông qua các payload CERT và AUTH, hai bên trao đổi khóa công khai và chứng minh quyền sở hữu khóa bí mật tương ứng. Hệ thống chỉ cho phép thiết lập đường ống bảo mật sau khi xác minh danh tính thiết bị khớp với chính sách truy cập và chứng chỉ chưa bị thu hồi.

Mô hình này sở hữu tính mở rộng vượt trội nhờ khả năng cô lập rủi ro: quản trị viên có thể vô hiệu hóa nhanh chóng một chứng chỉ nhân viên khi họ nghỉ việc. Đổi lại, tổ chức phải vận hành PKI nội bộ nghiêm ngặt. Việc cấp sai chứng chỉ, chậm trễ cập nhật danh sách thu hồi hay lưu trữ khóa kém an toàn tại gateway đều trực tiếp làm sụp đổ ranh giới bảo mật mạng.

5.5 Code Signing

Hệ điều hành cần công cụ để đánh giá một phần mềm đến từ đâu và có bị thay đổi không, bởi kênh truyền tải thông thường không chống lại việc can thiệp trên server phản chiếu (mirror) hay đánh tráo gói tin. Code signing khắc phục lỗ hổng này bằng cách ép buộc nhà phát hành phải ký trực tiếp lên phần mềm, driver hoặc script [20], [21].

Dù chia sẻ nền tảng mật mã với TLS, chứng chỉ ký mã bị khống chế bởi trường ExtendedKeyUsage hoàn toàn khác rẽ nhánh từ mục đích ban đầu. Khi thực thi phần mềm, hệ thống quét chữ ký, dựng chuỗi tới trust store và hiển thị tên nhà cung cấp. Tương tự chữ ký tài liệu, mã nguồn luôn cần đính kèm timestamp để duy trì giá trị hợp lệ của phần mềm ngay cả khi chứng chỉ phát hành hết hạn vào nhiều năm sau.

Sự toàn vẹn của mô hình này phụ thuộc tuyệt đối vào việc bảo vệ khóa. Nếu private key bị lộ hoặc đường ống phân phối CI/CD bị xâm nhập, hacker có thể phát tán mã độc dưới vỏ bọc một phần mềm hoàn toàn hợp pháp. Khi sự cố xảy ra, các CA bắt buộc phải thực thi thu hồi khẩn cấp, gây đứt gãy tức thời đối với mọi phần mềm hợp lệ khác dùng chung khóa ký đó.

5.6 eGovernment Việt Nam

Ở quy mô quốc gia, eGovernment đối mặt với bài toán liên thông: làm sao xác thực hàng triệu công dân, doanh nghiệp và văn bản hành chính một cách đồng nhất. Trang thông tin của Trung tâm Chứng thực điện tử quốc gia (NEAC) quy định rõ hệ thống này xoay quanh chứng chỉ công cộng, hạ tầng chữ ký số và các cơ chế kiểm tra văn bản thống nhất [22].

Trong mô hình ký số từ xa đang định hình thực tiễn tại Việt Nam, khóa bí mật không nằm trên thiết bị người dùng mà được cách ly tại hạ tầng bảo mật của CA. Người dùng chỉ cần xác thực qua phương thức an toàn để kích hoạt phiên ký. Cổng dịch vụ công khi tiếp nhận hồ sơ sẽ đóng vai trò verifier: nó không chỉ quét tính toàn vẹn của chữ ký XML/PDF, mà còn phải truy vấn trạng thái thu hồi chứng chỉ tại đúng thời điểm đóng dấu timestamp.

Điểm yếu dễ khai thác nhất nằm ở sự thiếu đồng bộ. Nếu cổng dịch vụ công của một địa phương áp dụng mức kiểm tra lỏng lẻo (chỉ kiểm tra định dạng chữ ký mà phớt lờ trạng thái CRL/OCSP), hệ thống sẽ dễ dàng tiếp nhận và lưu trữ các văn bản đóng dấu bởi chứng chỉ đã bị đình chỉ. Thêm vào đó, nếu phương thức xác thực kích hoạt ký từ xa yếu kém, chữ ký dù hoàn hảo về mặt mật mã vẫn có thể không phản ánh đúng ý chí thực sự của công dân.

Chapter 6

Các hệ thống PKI mã nguồn mở

Hệ sinh thái PKI nguồn mở không tồn tại một "công cụ vạn năng" giải quyết mọi bài toán. Thay vào đó, nó phân mảnh thành các lớp giải pháp chuyên biệt, từ những bộ thư viện mật mã lõi cho đến các nền tảng quản trị danh tính quy mô doanh nghiệp. Việc lựa chọn công cụ phụ thuộc hoàn toàn vào mục tiêu vận hành.

6.1 Tiêu chí phân loại

Dựa trên vai trò trong hệ thống, các công cụ PKI thường được chia thành năm nhóm chính:

Cryptographic Toolkit: Các tiện ích dòng lệnh và thư viện dùng để can thiệp trực tiếp vào cấu trúc mật mã (tạo khóa, ký CSR, parse chứng chỉ). OpenSSL là nền tảng tiêu biểu nhất.

Local Management: Các phần mềm giao diện đồ họa hướng tới người dùng cá nhân hoặc môi trường lab (ví dụ: XCA [23]), lưu trữ cấu trúc CA gọn nhẹ trong cơ sở dữ liệu cục bộ.

ACME Client & Public CA: Nhóm công cụ sinh ra để giải quyết bài toán cấp phát chứng chỉ tự động trên diện rộng cho Web PKI, dẫn đầu bởi Let's Encrypt và Certbot [17], [24], [25].

CA Platform: Các hệ thống cấp doanh nghiệp như EJBCA hay Dogtag [26], [27]. Chúng quản lý vòng đời chứng chỉ thông qua hệ thống chính sách (profile), tích hợp audit, HSM và phân quyền quản trị nghiêm ngặt.

Internal PKI & Secret Management: Các cỗ máy phát hành chứng chỉ nội bộ vòng đời cực ngắn (short-lived), hoạt động hoàn toàn qua API để phục vụ xác thực giao tiếp giữa các vi dịch vụ (mTLS). HashiCorp Vault PKI là đại diện nổi bật [28].

Một công cụ xuất sắc khi học tập chưa chắc đã đáp ứng được yêu cầu audit của một CA thực tế. Ngược lại, việc triển khai một nền tảng enterprise chỉ để cấp vài chứng chỉ nội bộ là sự lãng phí về tài nguyên và chi phí vận hành.

Hình 6.1 phân lớp hệ sinh thái mã nguồn mở theo mục tiêu sử dụng.

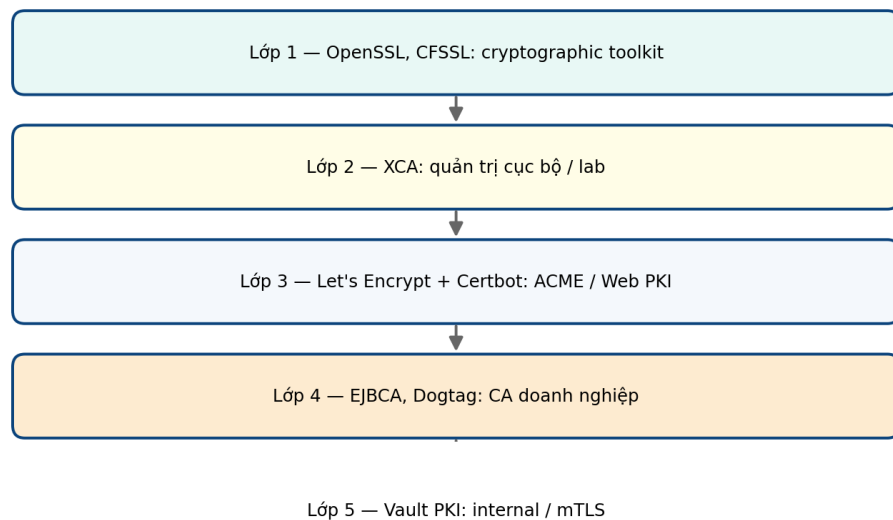


Figure 6.1: Phân lớp hệ sinh thái PKI mã nguồn mở theo quy mô và mục tiêu triển khai

6.2 OpenSSL

OpenSSL đóng vai trò "con dao Thụy Sĩ" trong hệ sinh thái mật mã. Dù không phải là một nền tảng CA hoàn chỉnh, nó cung cấp các module nền tảng để tương tác trực tiếp với cấu trúc X.509.

Lệnh `openssl x509` cho phép bóc tách và hiển thị từng trường dữ liệu của chứng chỉ, chuyển đổi định dạng (PEM/DER) hoặc ký trực tiếp một CSR [29]. Trong khi đó, lệnh `openssl verify` là công cụ đặc lực để gỡ lỗi quá trình dựng chuỗi tín nhiệm, cho phép chỉ định rõ trust anchor, đối chiếu hostname, email hay mục đích sử dụng [30].

```

1 openssl x509 -in server.crt -text -noout
2 openssl verify -CAfile ca-chain.pem server.crt
3 openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -out
   server.crt
  
```

Listing 6.1: Thao tác cấu trúc chứng chỉ bằng lệnh OpenSSL

Điểm yếu chí mạng của OpenSSL khi đóng vai trò CA là sự vắng bóng của tầng quản trị. Mọi thao tác đều thủ công hoặc phải bọc qua các bash script. Nó không có khái niệm

phân quyền quản trị, không có portal duyệt yêu cầu (RA), và thiếu vắng cơ chế ghi log tập trung phục vụ kiểm toán.

6.3 XCA

Để khắc phục rào cản dòng lệnh khô khan của OpenSSL, XCA mang đến một giao diện đồ họa trực quan để quản trị kho chứng chỉ cục bộ. Toàn bộ thông tin từ khóa riêng, CSR, chứng chỉ, CRL cho đến các template đều được tổ chức gọn gàng trong một cơ sở dữ liệu duy nhất (SQLite, MySQL hoặc PostgreSQL) [23].

Sức mạnh của XCA nằm ở khả năng trực quan hóa. Người dùng có thể dễ dàng khởi tạo một Root CA, phân nhánh các Sub-CA và quan sát toàn bộ chuỗi tín nhiệm dưới dạng cây thư mục. Nó lý tưởng cho mục đích học tập hoặc quản lý mạng nội bộ quy mô nhỏ.

Tuy nhiên, giới hạn của XCA hoàn toàn tương đồng với OpenSSL: nó là công cụ cá nhân (local management). Nó thiếu API chuẩn hóa để tích hợp tự động, không có quy trình thẩm định (enrollment workflow) và không thể mở rộng thành một dịch vụ cấp phát chứng chỉ tập trung.

6.4 EJBCA

Khác biệt hoàn toàn với OpenSSL hay XCA, EJBCA là một nền tảng quản trị vòng đời chứng chỉ cấp doanh nghiệp thực thụ [26]. Trái tim của EJBCA không nằm ở thuật toán tạo khóa, mà ở hệ thống chính sách khắt khe kiểm soát việc phát hành.

Sự chặt chẽ này thể hiện qua việc phân tách hai lớp profile:

- **Certificate Profile:** Định nghĩa "hình hài" của chứng chỉ được cấp ra (thuật toán, độ dài khóa, `KeyUsage`, `ExtendedKeyUsage`, tích hợp Certificate Transparency) [31].
- **End Entity Profile:** Quy định "ai" được phép xin cấp chứng chỉ (cấu trúc Subject DN bắt buộc) và ràng buộc họ vào những Certificate Profile cụ thể nào [32].

Về mặt giao tiếp, EJBCA bộc lộ kiến trúc hiện đại qua hệ thống REST API đa dạng, bao phủ từ quản trị CA, duyệt yêu cầu đến trích xuất cấu hình [33]. Hơn thế nữa, nó hỗ trợ giao thức ACME đầy đủ các challenge phổ biến (`http-01`, `dns-01`, `tls-alpn-01`) để tích hợp trơn tru với các workload tự động [34].

Trong khâu vận hành, EJBCA cung cấp sẵn OCSP Responder độc lập [35] và hỗ trợ kết nối trực tiếp với thiết bị mã hóa cứng (HSM). Việc bảo vệ khóa bí mật của Root CA trong HSM thay vì trên ổ cứng thông thường là lần ranh phân biệt giữa một hệ thống thử nghiệm và một hạ tầng khóa công khai đạt chuẩn an toàn thông tin quốc gia.

6.5 Let's Encrypt và Certbot

Nếu EJBCA giải quyết bài toán quản trị tập trung cho mạng nội bộ, thì Let's Encrypt lại làm một cuộc cách mạng trên không gian Web PKI công cộng. Được vận hành bởi Internet Security Research Group, đây là một CA công cộng cấp phát chứng chỉ TLS hoàn toàn miễn phí [24].

Linh hồn của Let's Encrypt là giao thức ACME (RFC 8555) và công cụ Certbot [25]. Thay vì phải thao tác xin cấp thủ công, Certbot chứng minh quyền sở hữu tên miền với CA thông qua các bộ giải đố tự động (challenge):

- **HTTP-01:** Tạo một file ngẫu nhiên tại `/.well-known/acme-challenge/` và phản hồi CA qua cổng 80 [36].
- **DNS-01:** Tạo bản ghi TXT tại `_acme-challenge`. Đây là phương pháp duy nhất hỗ trợ cấp chứng chỉ wildcard [36].

Lợi ích lớn nhất của Certbot là khả năng gia hạn im lặng (silent renewal). Lệnh `certbot renew` thường được lên lịch chạy ngầm qua cronjob, tự động xin cấp lại trước khi chứng chỉ hết hạn và kích hoạt các hook để tải lại cấu hình web server [25].

Tuy nhiên, quản trị viên cần hết sức chú ý đến các giới hạn tần suất (rate limit) của Let's Encrypt: việc kích bản tự động bị lỗi và liên tục xin cấp lại một tên miền quá nhiều lần có thể dẫn tới việc bị khóa tài khoản hoặc khóa tên miền trong nhiều ngày [37].

6.6 Vault PKI

HashiCorp Vault PKI Secrets Engine đại diện cho triết lý thiết kế bảo mật vi dịch vụ (microservices). Thay vì xin cấp chứng chỉ thời hạn hàng năm từ một CA truyền thống, các dịch vụ gọi API đến Vault để lấy một chứng chỉ có vòng đời (TTL) chỉ tính bằng giờ, thậm chí bằng phút [28].

Quá trình cấp phát được ràng buộc bởi hệ thống Role, quy định chặt chẽ domain, subdomain và thời gian sống tối đa của chứng chỉ. Khi gọi endpoint `issue`, client nhận về trọn bộ chứng chỉ, khóa riêng và chuỗi CA để lập tức sử dụng [38].

Triết lý TTL siêu ngắn mang lại ưu thế tuyệt đối về mặt vận hành: nó gần như triệt tiêu nhu cầu duy trì hệ thống thu hồi (CRL/OCSP) vốn rất nặng nề. Kẻ tấn công nếu đánh cắp được khóa phiên cũng chỉ có thể lợi dụng trong thời gian rất ngắn trước khi chứng chỉ tự bốc hơi. Mặc dù vậy, Vault PKI chỉ được các hệ thống nội bộ tin cậy; nó không thể dùng để cấp chứng chỉ TLS công cộng hướng tới thiết bị của người dùng cuối.

6.7 OAuth/OIDC và PKI

Dù OAuth 2.0 (ủy quyền) và OpenID Connect (định danh) không phải là hạ tầng PKI, chúng sử dụng các thành tố mật mã rất tương đồng. OIDC đóng gói các thông tin xác thực vào JSON Web Token (JWT), và tính toàn vẹn của token này được bảo đảm bằng chữ ký số (JWS) [39], [40], [41].

Thay vì dùng chứng chỉ X.509 truyền thống, IdP công bố khóa công khai dưới dạng tập hợp JSON Web Key Set (JWKS) tại một endpoint cố định [42]. Dịch vụ tài nguyên sẽ kéo bộ khóa này về, đối chiếu trường `kid` (Key ID) để tìm đúng khóa và kiểm chứng chữ ký trên token.

Điểm giao thoa sâu sắc nhất giữa hệ thống định danh này và PKI là Mutual TLS (mTLS) trong OAuth. Theo RFC 8705, access token có thể được neo chặt (bound) vào một chứng chỉ client X.509. Khi đó, tài nguyên bị khóa kép: client không chỉ phải nộp token hợp lệ, mà còn phải thiết lập kênh mTLS bằng đúng chứng chỉ đã đăng ký, ngăn chặn triệt để rủi ro bị kẻ gian đánh cắp và lạm dụng token [43].

6.8 Dogtag và FreeIPA

Dogtag Certificate System là một CA platform mã nguồn mở cấp doanh nghiệp, cung cấp đầy đủ dịch vụ quản lý vòng đời chứng chỉ, profile, OCSP và SCEP [27]. Trong khi EJBCA mang tính đa nền tảng, Dogtag lại gắn kết chặt chẽ với hệ sinh thái Linux doanh nghiệp (đặc biệt là họ hệ điều hành Red Hat).

Sự kết hợp này phát huy tối đa sức mạnh khi Dogtag được nhúng vào làm lõi CA cho FreeIPA. Khác với CA truyền thống, FreeIPA là hệ thống quản trị định danh (Identity Management) toàn diện, bao bọc Kerberos, LDAP, DNS, user và host vào chung một miền duy nhất [44]. Việc tích hợp sẵn Dogtag cho phép quản trị viên tự động hóa hoàn toàn luồng bảo mật: một máy chủ Linux khi gia nhập mạng nội bộ sẽ tự động nhận danh tính Kerberos, đồng thời song song được cấp chứng chỉ X.509 để mã hóa kênh truyền nội bộ, tất cả được điều phối nhịp nhàng từ một điểm tập trung.

6.9 So sánh hệ sinh thái

Table 6.1: So sánh một số công cụ và hệ thống PKI mã nguồn mở

Công cụ	Nhóm	Vai trò chính	Môi trường	CA	API	Thu hồi	Định danh	Độ phức tạp	Giới hạn chính
OpenSSL	Toolkit	Thao tác khóa, CSR, chứng chỉ	Local, script	Rất cơ bản	CLI	Qua CRL khi cấu hình	Không	Thấp	Không có RA workflow hoặc audit tập trung.

Công cụ	Nhóm	Vai trò chính	Môi trường	CA	API	Thu hồi	Định danh	Độ phức tạp	Giới hạn chính
XCA	Local management	Quản lý CA và certificate store cục bộ	Lab, học tập	CA cục bộ	Thấp	Có CRL	Không	Thấp đến vừa	Không phải enterprise CA hoặc API-first system.
EJBCA	CA platform	Quản trị CA có profile và lifecycle	CA nội bộ, CA có policy	Đầy đủ	REST, ACME	OCSP, CRL	RA, OAuth cho REST	Cao	Cần cấu hình profile, quyền và vận hành khóa.
Let's Encrypt/Certbot	Public ACME	Tự động hóa TLS public	Web PKI công cộng	CA công cộng	ACME, plugin	ACME revocation	Domain validation	Vừa	Phụ thuộc domain control và rate limit.
Vault PKI	Internal PKI	Cấp chứng chỉ nội bộ ngắn hạn	Service nội bộ, mTLS	CA nội bộ	HTTP API, role	CRL, TTL ngắn	Vault auth/policy	Vừa	Không thay public CA hoặc CA enterprise.
OAuth/OIDC stack	Identity/auth	Xác minh token và identity	IdP, API	Không phải CA	JWKS, token validation	Không quản lý chứng chỉ	Rất mạnh	Vừa đến cao	Quản lý token, không quản lý PKI.
Dogtag/FreeIPA	Enterprise identity + PKI	CA và certificate trong identity domain	Linux enterprise	Đầy đủ trong Dogtag	Web, CLI	CRL, OCSP	LDAP, Kerberos	Cao	Phức tạp và gắn với hệ sinh thái Linux.

Chapter 7

Thách thức và xu hướng

Dù các thuật toán mật mã liên tục tiến hóa, vai trò cốt lõi của PKI trong việc đóng đinh khóa công khai vào danh tính vẫn không thể thay thế. Tuy nhiên, khi chuyển từ quy mô hàng nghìn người dùng lên hàng tỷ thiết bị IoT và vi dịch vụ, nút thắt bảo mật không còn nằm ở toán học, mà dịch chuyển hoàn toàn sang quy trình vận hành: kiểm soát vòng đời, quản lý rủi ro từ CA, và tự động hóa.

7.1 Thách thức hiện tại

Sự đứt gãy hệ thống tồi tệ nhất thường không đến từ hacker, mà đến từ một chứng chỉ bị lãng quên cho tới khi hết hạn. Ở quy mô lớn, việc thiếu một hệ thống kiểm kê (inventory) minh bạch khiến việc xoay vòng khóa (key rotation) trở thành cơn ác mộng thủ công. Tổ chức thường không biết chính xác mình đang sở hữu bao nhiêu chứng chỉ, do ai cấp và nằm ở đâu trong hạ tầng.

Bên cạnh đó, cơ chế thu hồi (revocation) chưa bao giờ hoàn hảo. Danh sách thu hồi CRL [5] thường xuyên phình to và có độ trễ cập nhật lớn, trong khi giao thức OCSP [6] lại tạo ra điểm nghẽn cổ chai (bottleneck) và đặt ra câu hỏi khó: client nên làm gì khi máy chủ OCSP sập (soft-fail hay hard-fail)?

Cao hơn nữa, bản thân mô hình phân cấp CA là một điểm yếu chí mạng (single point of failure). Khi một CA công cộng bị xâm nhập hoặc cấp phát sai nguyên tắc, hậu quả sẽ lan rộng trên toàn cầu trước khi các trình duyệt kịp phản ứng bằng cách cập nhật trust store. CA/B Forum đã phải thiết lập Baseline Requirements [4] nhằm siết chặt quy trình, nhưng nó chỉ là rào chắn pháp lý chứ không loại bỏ được hoàn toàn rủi ro kỹ thuật.

7.2 Tự động hóa và giám sát

Để giải quyết bài toán quy mô, thế giới đang chuyển dịch mạnh mẽ sang tự động hóa và rút ngắn thời gian sống của chứng chỉ. Giao thức ACME [17] đã chứng minh rằng thao

tác xác minh tên miền và cấp phát chứng chỉ có thể diễn ra hoàn toàn không cần bàn tay con người. Kết hợp với xu hướng chứng chỉ ngắn hạn (short-lived certificate), tổ chức có thể giới hạn tối đa cửa sổ rủi ro (window of vulnerability) nếu khóa bị đánh cắp.

Tuy nhiên, tự động hóa mù quáng là thảm họa. Nếu thiếu hệ thống phân quyền (RBAC) và chính sách (policy) khắt khe, việc tự động hóa chỉ đơn giản là khuếch đại tốc độ cấp phát sai lầm.

Trên Internet công cộng, Web PKI được gia cố thêm lớp phòng ngự Certificate Transparency (CT) [15]. Bằng cách yêu cầu mọi chứng chỉ phát hành phải được ghi vào các log công khai (append-only) dựa trên cấu trúc Merkle Tree, CT loại bỏ hoàn toàn khả năng CA âm thầm cấp phát chứng chỉ giả mạo cho các tên miền nhạy cảm mà chủ sở hữu không hay biết.

Hình 7.1 mô tả ý tưởng CT: CA ghi chứng chỉ vào log công khai; chủ tên miền và trình duyệt có thể giám sát các bản ghi bất thường.

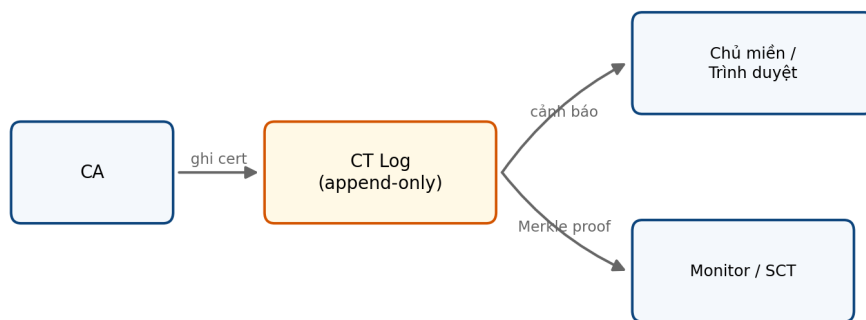


Figure 7.1: Tổng quan Certificate Transparency và vai trò log công khai

7.3 PKI trong hệ thống hiện đại

Định nghĩa "chủ thể" (subject) trong PKI đang dịch chuyển từ con người sang máy móc (machine identity). Hàng triệu container, vi dịch vụ (microservice) và thiết bị IoT liên tục được tạo ra và tiêu hủy trong vài giờ. Chúng đòi hỏi danh tính mật mã tức thời để giao tiếp an toàn.

Đặc biệt trong trào lưu kiến trúc Zero Trust, ranh giới mạng nội bộ không còn được coi là an toàn. Mutual TLS (mTLS) trở thành tiêu chuẩn bắt buộc: mọi dịch vụ phải xuất trình chứng chỉ hợp lệ khi nói chuyện với nhau, bất kể chúng nằm ở đâu. Sự thay đổi này ép buộc các tổ chức phải xây dựng Internal PKI có khả năng cấp phát tốc độ cao, tích hợp trực tiếp vào hệ thống Secret Management thay vì duy trì các CA thủ công như thập kỷ trước.

7.4 Sẵn sàng cho kỷ nguyên hậu lượng tử (Post-quantum)

Hiểm họa máy tính lượng tử đang đe dọa trực tiếp nền móng của RSA và thuật toán đường cong elliptic (ECC). Dù NIST đã chốt danh sách các thuật toán hậu lượng tử (PQC) vào năm 2024 [45], bài toán tích hợp chúng vào PKI lại vô cùng gian nan. Các thuật toán mới thường có kích thước khóa và chữ ký khổng lồ, đòi hỏi phải đập đi xây lại hàng loạt tiêu chuẩn từ cấu trúc X.509, phần cứng HSM cho đến các giao thức bắt tay TLS.

Thay vì chờ đợi "ngày phán xét" (Y2Q), ngành bảo mật đang chuyển dịch sang khái niệm *Crypto-Agility* (Tính linh hoạt mật mã). Một hạ tầng PKI hiện đại phải được thiết kế sao cho việc thay thế thuật toán cốt lõi diễn ra trơn tru như việc thay lốp xe đua trên trạm dừng, không làm gián đoạn chuỗi cung ứng chứng chỉ đang vận hành.

Chapter 8

Kết luận

8.1 Tổng kết

Hạ tầng khóa công khai (PKI) không đơn thuần là một bản phác thảo mật mã tĩnh lặng trên giấy. Bắt đầu từ những thuật toán mã hóa nền tảng, nó đã phát triển thành một hệ sinh thái khổng lồ định hình kiến trúc bảo mật toàn cầu.

Dù ở hình thái bảo vệ lưu lượng HTTPS web, ký duyệt tài liệu pháp lý, hay nhúng sâu vào môi trường Zero Trust dưới dạng vi dịch vụ, hạt nhân của hệ thống vẫn xoay quanh một mục tiêu duy nhất: neo giữ niềm tin có thể xác minh giữa các thực thể ẩn danh.

Sự tiến hóa của PKI hiện nay đã rời xa các tranh luận về thuật toán cơ sở để đối mặt với thực tế khốc liệt của vận hành: bài toán mở rộng quy mô, kiểm soát vòng đời chứng chỉ ngắn hạn và sự chuẩn bị trước bình minh của kỷ nguyên máy tính lượng tử. PKI, tựu trung lại, là sự kết hợp chặt chẽ giữa tính chính xác của toán học, sự khắt khe của chính sách kiểm định và tự động hóa vận hành.

8.2 Ý nghĩa và lời cảm ơn

Quá trình thực hiện tiểu luận giúp nhóm phá vỡ những hiểu lầm ban đầu về bảo mật ứng dụng. Thay vì coi chứng chỉ số như một hộp đen cấu hình, nhóm đã nắm bắt được quy trình neo tín nhiệm từ Root CA đến thực thể cuối (End-Entity), cũng như thấu hiểu sức nặng của bài toán thu hồi và tự động hóa. Những kiến thức thu nhận được không chỉ dừng ở lý thuyết, mà trực tiếp phục vụ cho việc xây dựng kiến trúc bảo mật cho các dự án phần mềm sau này.

Chúng em xin gửi lời cảm ơn chân thành tới PGS. TS. Nguyễn Linh Giang. Sự hướng dẫn và định hướng chuyên môn sâu sắc của cô là kim chỉ nam để nhóm hoàn thành trọn vẹn báo cáo này.

Tài liệu tham khảo

- [1] J. G. Steiner, B. C. Neuman, and J. I. Schiller, “Kerberos: An authentication service for open network systems,” in *Proceedings of the USENIX Winter Conference*, 1988, pp. 191–202.
- [2] R. L. Rivest, A. Shamir, and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978. DOI: [10.1145/359340.359342](https://doi.org/10.1145/359340.359342)
- [3] P. R. Zimmermann, *PGP: Source code and internals*, 1995.
- [4] CA/Browser Forum, *Baseline requirements for the issuance and management of publicly-trusted certificates*, 2023. [Online]. Available: <https://cabforum.org/baseline-requirements-documents/>
- [5] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, “Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile,” IETF, RFC 5280, 2008. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5280>
- [6] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, “X.509 Internet Public Key Infrastructure Online Certificate Status Protocol – OCSP,” IETF, RFC 6960, 2013. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6960>
- [7] D. E. 3rd, “Transport Layer Security (TLS) Extensions: Extension Definitions,” IETF, RFC 6066, 2011, Section 8: Certificate Status Request (OCSP Stapling). [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6066>
- [8] P. Saint-Andre and J. Hodges, “Representation and Verification of Domain-Based Application Service Identity within Internet Public Key Infrastructure Using X.509 Certificates in the Context of Transport Layer Security (TLS),” IETF, RFC 6125, 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6125>
- [9] M. Nystrom and B. Kaliski, “PKCS #10: Certification Request Syntax Specification Version 1.7,” IETF, RFC 2986, 2000. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2986>

- [10] CA/Browser Forum, *Guidelines for the issuance and management of extended validation certificates*, 2023. [Online]. Available: <https://cabforum.org/extended-validation/>
- [11] J. Schaad, B. Ramsdell, and S. Turner, “Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 4.0 Message Specification,” IETF, RFC 8551, 2019. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8551>
- [12] R. Housley, “Cryptographic Message Syntax (CMS),” IETF, RFC 5652, 2009. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5652>
- [13] K. Moriarty, M. Nystrom, S. Parkinson, A. Rusch, and M. Scott, “PKCS #12: Personal Information Exchange Syntax v1.1,” IETF, RFC 7292, 2014. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7292>
- [14] B. Laurie, A. Langley, and E. Kasper, “Certificate Transparency,” IETF, RFC 6962, 2013. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6962>
- [15] B. Laurie, A. Langley, E. Kasper, E. Messeri, and R. Stradling, “Certificate Transparency Version 2.0,” IETF, RFC 9162, 2021. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9162>
- [16] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” IETF, RFC 8446, 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8446>
- [17] R. Barnes, J. Hoffman-Andrews, D. McCarney, and J. Kasten, “Automatic Certificate Management Environment (ACME),” IETF, RFC 8555, 2019. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8555>
- [18] C. Adams, P. Cain, D. Pinkas, and R. Zuccherato, “Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP),” IETF, RFC 3161, 2001. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3161>
- [19] C. Kaufman, P. Hoffman, Y. Nir, P. Eronen, and T. Kivinen, “Internet Key Exchange Protocol Version 2 (IKEv2),” IETF, RFC 7296, 2014. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7296>
- [20] Microsoft, *Cryptography tools – win32 apps*, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/seccrypto/cryptography-tools>
- [21] Apple Developer Documentation, *Code signing services*, 2026. [Online]. Available: <https://developer.apple.com/documentation/security/code-signing-services>
- [22] Trung tâm Chứng thực điện tử quốc gia, *Trang thông tin điện tử trung tâm chứng thực điện tử quốc gia*, 2026. [Online]. Available: <https://neac.gov.vn>
- [23] C. Hohnstaedt, *Xca: Certificate and key management*, 2026. [Online]. Available: <https://hohnstaedt.de/xca/>

- [24] Let's Encrypt, *Documentation*, 2026. [Online]. Available: <https://letsencrypt.org/docs/>
- [25] Electronic Frontier Foundation, *Certbot user guide*, 2026. [Online]. Available: <https://eff-certbot.readthedocs.io/en/stable/using.html>
- [26] Keyfactor, *Ejbca documentation*, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/>
- [27] Dogtag PKI Team, *Dogtag pki documentation*, 2026. [Online]. Available: https://www.dogtagpki.org/wiki/PKI_Main_Page
- [28] HashiCorp, *Pki secrets engine*, 2026. [Online]. Available: <https://developer.hashicorp.com/vault/docs/secrets/pki>
- [29] OpenSSL Project, *openssl-x509: Certificate display and signing command*, 2026. [Online]. Available: <https://docs.openssl.org/master/man1/openssl-x509/>
- [30] OpenSSL Project, *openssl-verify: Certificate verification command*, 2026. [Online]. Available: <https://docs.openssl.org/master/man1/openssl-verify/>
- [31] Keyfactor, *Ejbca certificate profiles overview*, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/certificate-profiles-overview>
- [32] Keyfactor, *Ejbca end entity profiles overview*, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/end-entity-profiles-overview>
- [33] Keyfactor, *Ejbca rest interface*, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/ejbca-rest-interface>
- [34] Keyfactor, *Ejbca acme*, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/acme>
- [35] Keyfactor, *Ejbca ocsp*, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/ocsp>
- [36] Let's Encrypt, *Challenge types*, 2026. [Online]. Available: <https://letsencrypt.org/docs/challenge-types/>
- [37] Let's Encrypt, *Rate limits*, 2026. [Online]. Available: <https://letsencrypt.org/docs/rate-limits/>
- [38] HashiCorp, *Build your own certificate authority with vault*, 2026. [Online]. Available: <https://developer.hashicorp.com/vault/docs/secrets/pki/quick-start-root-ca>
- [39] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, *OpenID Connect Core 1.0*, 2014. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html

- [40] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF, RFC 7519, 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [41] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Signature (JWS),” IETF, RFC 7515, 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7515>
- [42] M. Jones, “JSON Web Key (JWK),” IETF, RFC 7517, 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7517>
- [43] B. Campbell, J. Bradley, N. Sakimura, and T. Lodderstedt, “OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens,” IETF, RFC 8705, 2020. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8705>
- [44] FreeIPA Project, *About freeipa*, 2026. [Online]. Available: <https://www.freeipa.org/page/About>
- [45] National Institute of Standards and Technology, “Post-Quantum Cryptography: FIPS 203, FIPS 204, FIPS 205,” NIST, Tech. Rep., 2024. [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography>